

Tema

MANUAL DE CONFIGURACIÓN SISTEMA CONVERSIÓN UNIDADES MONSTER CON RESTFUL SIN BDD EN JAVA

Autores

ALEJANDRO CAETANO FLORES MAYA

ANTHONY ALAIN MORALES CARVAJAL

LEONARDO VINICIO NARVÁEZ CRIOLLO

Tutor

Ing. Eduardo Mauricio Campaña Ortega

MIS.MDU.CCNA. CCIA.

PhD. (c) Ingeniería de Software

PhD. (c) Seguridad Información

Fecha

28/10/2026

ÍNDICE DE CONTENIDOS

1 INTRODUCCIÓN 5
2 OBJETIVOS 5
2.1 Objetivo General 5
2.2 Objetivos Específicos 5
3 MARCO TEÓRICO 6
3.1 Web Services RESTful, HTTP y JSON 6
3.2 Arquitectura Orientada a Recursos (ROA) y REST vs SOA/SOAP 6
3.3 Patrón de Arquitectura Modelo-Vista-Controlador (MVC) 7
3.4 Servidor Payara 6 y Jakarta REST (JAX-RS) 7
4 DESARROLLO DEL APLICATIVO 7
4.1 Servidor Web RESTful 7
4.1.1 Paso 1: Creación de la Instancia de Servidor Payara en el IDE 8
4.1.2 Paso 2: Selección de Versión y Directorio de Instalación 8
4.1.3 Paso 3: Configuración de Dominio y Puertos de Red 8
4.1.4 Paso 4: Creación del Proyecto Tipo Aplicación Web 9
4.1.5 Paso 5: Parametrización de Metadatos del Proyecto 9
4.1.6 Paso 6: Asociación del Servidor y Plataforma Jakarta EE 10 10
4.1.7 Paso 7: Estructuración de Paquetes del Servidor 10
4.1.8 Paso 8: Configuración de la Aplicación REST (RestApplication.java) 11
4.1.9 Paso 9: Implementación del Filtro Global de CORS (CorsFilter.java) 12
4.1.10 Paso 10: Implementación del Endpoint Central de Compatibilidad (WSConversorUnidadesRest.java) 13
4.1.11 Paso 11: Implementación de Recursos REST Dedicados (/api/auth y /api/conversor) 17
4.1.12 Paso 12: Implementación de la Capa de Dominio y Conversión 24
4.1.13 Paso 13: Implementación del Manejador de Seguridad y Sesión en Memoria (SIN BDD) 30
4.1.14 Paso 14: Despliegue en Servidor y Comprobación de Endpoints REST 35
4.2 Cliente Consola 35
4.2.1 Paso 1: Creación del Proyecto Maven en NetBeans 35
4.2.2 Paso 2: Configuración del Cliente HTTP y Dependencias Maven 36
4.2.3 Paso 3: Estructuración Modular de Paquetes (MVC Estricto) 36
4.2.4 Paso 4: Implementación de la Capa de Modelo (ConversorService.java) 37
4.2.5 Paso 5: Implementación de Controladores y Vista de Consola 40
4.3 Cliente Escritorio 47
4.3.1 Paso 1: Creación del Proyecto Gráfico en NetBeans 47
4.3.2 Paso 2: Configuración del Cliente HTTP y Modelos DTO 48
4.3.3 Paso 3: Estructuración de Paquetes MVC 48
4.3.4 Paso 4: Implementación de la Capa de Modelo (ConversorService.java) 49

4.3.5 Paso 5: Diseño de Vistas con Formularios Swing 52
4.3.6 Paso 6: Implementación del Controlador de Eventos (AppControlador.java) 53
4.4 Cliente Móvil 55
4.4.1 Paso 1: Creación y Configuración del Proyecto Android en Gradle 55
4.4.2 Paso 2: Estructuración Modular bajo el Patrón MVC 55
4.4.3 Paso 3: Implementación de la Capa de Modelo y Despacho REST Asíncrono 56
4.4.4 Paso 4: Diseño de Layouts XML 59
4.4.5 Paso 5: Implementación de Actividades y Controlador Móvil 68
4.5 Cliente Web 70
4.5.1 Paso 1: Creación del Proyecto Web Jakarta Faces en NetBeans 70
4.5.2 Paso 2: Configuración del Entorno JSF y Descriptores 71
4.5.3 Paso 3: Estructuración MVC en Jakarta Faces 72
4.5.4 Paso 4: Implementación de la Capa de Modelo (ConversorService.java) 72
4.5.5 Paso 5: Implementación de Managed Beans Controladores con CDI 76
4.5.6 Paso 6: Diseño de Vistas Web con Facelets (XHTML) 80
5 CONCLUSIONES 84
6 RECOMENDACIONES 84
7 REFERENCIAS BIBLIOGRÁFICAS 85

ÍNDICE DE ILUSTRACIONES

Figura 1 Creación de Servidor Payara en NetBeans 8
Figura 2 Configuración básica del servidor Payara 8
Figura 3 Configuración avanzada del servidor Payara 9
Figura 4 Creación de Proyecto Tipo Aplicación Web REST 9
Figura 5 Parámetros del proyecto servidor Maven 10
Figura 6 Selección de Servidor y perfil Jakarta EE 10 10
Figura 7 Estructura modular de paquetes del servidor REST 11
Figura 8 Configuración de RestApplication (@ApplicationPath) 11
Figura 9 Declaración de Endpoints RESTful con anotaciones JAX-RS 12
Figura 10 Implementación del filtro de CORS y seguridad en memoria 31
Figura 11 Endpoint RESTful y recursos JSON publicados en Payara Server 35
Figura 12 Creación del proyecto Maven para el Cliente Consola 36
Figura 13 Configuración del pom.xml y dependencias del Cliente Consola 36
Figura 14 Estructura de paquetes MVC del Cliente Consola en NetBeans 37
Figura 15 Ejecución del Cliente Consola (Inicio de sesión con banner Sullivan) 47
Figura 16 Ejecución de operaciones de conversión en Cliente Consola 47
Figura 17 Creación del Proyecto Java Swing en NetBeans 48
Figura 18 Configuración de módulos y dependencias en Cliente Escritorio 48
Figura 19 Estructuración de paquetes bajo el patrón MVC en Cliente Escritorio 48

Figura 20 Diseño visual del formulario de login en Swing (LoginView) 52
Figura 21 Diseño visual de la ventana principal del conversor (ConversorView) 53
Figura 22 Interfaz gráfica de Login ejecutada en Cliente Escritorio 54
Figura 23 Operación de conversión exitosa en Cliente Escritorio 55
Figura 24 Configuración de permisos de red en AndroidManifest.xml 55
Figura 25 Estructura modular de paquetes MVC en Cliente Móvil 56
Figura 26 Diseño visual de layout de login móvil (activity_login.xml) 60
Figura 27 Diseño visual de layout del conversor móvil (activity_conversor.xml) 60
Figura 28 Ejecución del Cliente Móvil (Login con imagen Sullivan) 69
Figura 29 Ejecución del Cliente Móvil (Conversión y resultados dinámicos) 70
Figura 30 Creación del proyecto Web Jakarta Faces en NetBeans 70
Figura 31 Descriptores de despliegue web.xml, beans.xml y faces-config.xml 71
Figura 32 Estructuración de paquetes MVC en Cliente Web 72
Figura 33 Interfaz Web de Login en navegador (login.xhtml) 83
Figura 34 Interfaz Web del Conversor con actualización reactiva Ajax (conversor.xhtml) ... 84

ÍNDICE DE TABLAS

Tabla 1 Configuración de la aplicación REST (RestApplication.java) 11
Tabla 2 Filtro global de CORS (CorsFilter.java) 12
Tabla 3 Declaración del Endpoint REST y autenticación (WSConversorUnidadesRest.java) 13
Tabla 4 Métodos de conversión expuestos en REST (WSConversorUnidadesRest.java) .. 15
Tabla 5 Recurso REST de Autenticación (AutenticacionResource.java) 17
Tabla 6 Recurso REST de Conversión (ConversorResource.java) 20
Tabla 7 Interfaz Conversor (IConversor.java) 24
Tabla 8 Conversor Longitud (ConversorLongitud.java) 25
Tabla 9 Conversor Masa (ConversorMasa.java) 25
Tabla 10 Conversor Temperatura (ConversorTemperatura.java) 26
Tabla 11 Servicio de Dominio Conversor (ServicioConversor.java) 28
Tabla 12 Mapeador de Unidades (UnidadMapper.java) 28
Tabla 13 Administrador de Credenciales en Memoria (AdministradorCredenciales.java) 31
Tabla 14 Administrador de Tokens de Sesión (AdministradorToken.java) 33
Tabla 15 Modelo de comunicación HTTP REST en Cliente Consola (ConversorService.java) 37
Tabla 16 Vista de Consola con Banner ASCII (ConsolaView.java) 40

Tabla 17 Controlador de Interfaz de Consola (ConsolaControlador.java) 43
Tabla 18 Controlador de Aplicación (AppControlador.java) 46
Tabla 19 Clase Principal de Ejecución (WSConversorConsoleClient.java) 46
Tabla 20 Capa de Modelo Cliente Escritorio (ConversorService.java) 49
Tabla 21 Controlador de Eventos de Escritorio (AppControlador.java) 53
Tabla 22 Modelo de comunicación REST asíncrona móvil (ConversorService.java) 56
Tabla 23 Diseño XML de la vista de autenticación móvil 60
Tabla 24 Diseño XML de la vista principal móvil 64
Tabla 25 Controlador de actividades móviles (AppControlador.java) 68
Tabla 26 Descriptores de despliegue web.xml 71
Tabla 27 Modelo de consumo REST en Cliente Web (ConversorService.java) 72
Tabla 28 Managed Bean reactivo con CDI (ConversorView.java) 76
Tabla 29 Formulario Facelets de autenticación web (login.xhtml) 80
Tabla 30 Interfaz Facelets del conversor web reactivo (conversor.xhtml) 81

1 INTRODUCCIÓN

Un servicio web basado en la arquitectura RESTful (Representational State Transfer) constituye un estilo arquitectónico moderno, ligero y altamente escalable para la comunicación e interoperabilidad de sistemas computacionales heterogéneos y distribuidos. Propuesto formalmente por Roy Fielding en el año 2000, el estilo REST desacopla de manera estricta el cliente del servidor y modela las capacidades operativas en torno al concepto fundamental de recurso. Cada recurso se encuentra unívocamente identificado mediante un Identificador Uniforme de Recursos (URI) y es manipulado a través de los verbos y métodos universales del protocolo de transferencia de hipertexto HTTP (GET, POST, PUT, DELETE), complementado con códigos de estado estándar de respuesta.

A diferencia del protocolo SOAP, el cual impone una pesada sobrecarga de procesamiento mediante envolturas XML rígidas y contratos WSDL obligatorios, las arquitecturas RESTful emplean de forma preferencial el formato de serialización JSON (JavaScript Object Notation). JSON proporciona un formato textual de datos estructurados sumamente ligero, de baja latencia y fácil serialización/deserialización, lo cual optimiza drásticamente el consumo de ancho de banda y maximiza la velocidad de respuesta en clientes de diversas plataformas (navegadores web, terminales de consola, entornos de escritorio e interfaces móviles).

En el desarrollo de servicios web REST bajo la plataforma Jakarta EE 10, la especificación estándar Jakarta RESTful Web Services 3.1 (JAX-RS) permite exponer recursos distribuidos mediante anotaciones declarativas (@ApplicationPath, @Path, @GET, @POST, @Produces, @Consumes). El despliegue de esta solución se efectúa sobre el servidor de aplicaciones Payara Server 6 Enterprise/Community, un entorno empresarial robusto y reactivo derivado de GlassFish. Payara provee soporte de clase empresarial para microservicios y APIs REST, garantizando alta concurrencia, administración declarativa y soporte nativo para filtros perimetrales de seguridad y políticas de Intercambio de Recursos de Origen Cruzado (CORS).

Asimismo, en este proyecto se implementa un modelo de autenticación y seguridad en memoria sin base de datos (SIN BDD), desacoplado y sustentado en tokens criptográficos (Bearer Tokens) generados y validados a través de estructuras concurrentes seguras (ConcurrentHashMap). Este esquema permite gobernar el acceso autorizado de cuatro plataformas cliente heterogéneas (consola interactiva, interfaz de escritorio en Java Swing, aplicación web en Jakarta Faces JSF 4.0 y aplicación móvil nativa Android) bajo una arquitectura MVC rigurosa y consistente.

2 OBJETIVOS

2.1 Objetivo General

Desarrollar un manual de configuración y desarrollo integral que describa de manera exhaustiva la construcción de un sistema conversor de unidades de medida (longitud, masa y temperatura) basado en el estilo arquitectónico RESTful y el patrón Modelo-Vista-Controlador (MVC), integrando cuatro clientes nativos (escritorio en Java Swing, consola en CLI, web en Jakarta Faces JSF 4.0 y móvil nativo en Android) con un servidor de servicios web REST implementado en Java Puro y Jakarta EE 10, sin base de datos (con autenticación mediante Bearer Tokens en memoria) y desplegado en Payara Server 6.

2.2 Objetivos Específicos

Describir detalladamente el diseño e implementación del servicio web RESTful utilizando la especificación Jakarta REST 3.1 (JAX-RS) sobre Payara Server 6, exponiendo endpoints

JSON bajo el prefijo /api/ y estructurando modularmente los conversores de longitud, masa y temperatura.
Implementar un mecanismo de seguridad y autenticación sin base de datos mediante Bearer Tokens en memoria gestionados concurrentemente, asegurando el intercambio seguro de tokens en las cabeceras HTTP Authorization y cargas útiles JSON.
Proporcionar guías paso a paso para el desarrollo y consumo del servicio RESTful en las cuatro plataformas cliente (escritorio, consola, web y móvil), detallando el desacoplamiento estricto del patrón MVC (exactamente 3 paquetes: model, view, controller) y la comunicación HTTP nativa con serialización JSON.
Explicar los fundamentos del estilo arquitectónico RESTful, las ventajas de la Arquitectura Orientada a Recursos (ROA) frente a SOAP, y el rendimiento de Payara Server 6 en términos de interoperabilidad, escalabilidad y procesamiento ligero de mensajes en entornos heterogéneos.

3 MARCO TEÓRICO

3.1 Web Services RESTful, HTTP y JSON

Un servicio web RESTful (Representational State Transfer) es un estilo de arquitectura de software para sistemas distribuidos hipermedia. En una arquitectura REST, toda la información que el sistema maneja se conceptualiza como un 'recurso'. Cada recurso es identificado por una URI unívoca y su estado se transfiere entre el cliente y el servidor mediante representaciones estándar, típicamente en formato JSON (JavaScript Object Notation).
Los principios axiomáticos del estilo REST son:

1. Cliente-Servidor Desacoplado: Separación estricta de responsabilidades entre la interfaz de usuario (cliente) y la lógica de negocio y almacenamiento (servidor).
2. Sin Estado (Stateless): Cada solicitud HTTP enviada por el cliente al servidor debe contener toda la información contextual necesaria para procesar la petición. El servidor no mantiene sesiones HTTP acopladas en su memoria de procesamiento para las llamadas REST, facilitando la escalabilidad horizontal.
3. Capacidad de Caché (Cacheable): Las respuestas del servidor deben definirse explícitamente como almacenables en caché o no, reduciendo el tráfico de red.
4. Interfaz Uniforme: Basada en URIs estandarizadas, manipulación de recursos mediante representaciones y uso de métodos HTTP semánticos (GET para recuperación idempotente, POST para creación/ejecución).
5. Sistema en Capas (Layered System): Permite interponer proxies, balanceadores de carga y filtros de seguridad intermedios sin que el cliente requiera cambios estructurales.

3.2 Arquitectura Orientada a Recursos (ROA) y REST vs SOA/SOAP

Mientras que la Arquitectura Orientada a Servicios (SOA) con SOAP se fundamenta en llamadas a procedimientos remotos (RPC) envueltas en complejos esquemas XML con contratos rígidos WSDL, la Arquitectura Orientada a Recursos (ROA) pone el foco en los datos y entidades expuestas. En ROA:

- Cada operación de conversión o autenticación se direcciona como un recurso específico (/api/auth/login, /api/conversor/longitud).
- No se requiere compilación ni descarga de contratos WSDL pesados en los clientes.

- Se reduce el tamaño de las tramas de red entre un 60% y un 80% comparado con SOAP/XML gracias a la concisión de JSON.
- El cliente utiliza llamadas HTTP nativas de Java (HttpURLConnection) o librerías estándar, manteniendo máxima compatibilidad multiplataforma.

3.3 Patrón de Arquitectura Modelo-Vista-Controlador (MVC)

El patrón arquitectónico MVC en este taller se implementa con un rigor estructural estricto de exactamente tres paquetes principales por cliente:

- Paquete model: Encapsula el modelo de datos, los enumeradores de magnitudes (UnidadLongitud, UnidadMasa, UnidadTemperatura), las sesiones activas y la clase ConversorService. La clase ConversorService concentra toda la infraestructura de red, ejecutando peticiones HTTP RESTful y parseando respuestas JSON sin acoplar detalles de transporte en el resto del cliente.
- Paquete view: Representa las interfaces visuales según la plataforma tecnológica: ConsolaView (terminal de texto CLI con banner ASCII), LoginView y ConversorView en Java Swing (ventanas JFrame), LoginView y ConversorView en Jakarta Faces JSF (páginas reactivas Facelets XHTML), y layouts XML en Android.
- Paquete controller: Implementado a través de AppControlador (y controladores auxiliares de flujo como ConsolaControlador), actúa como intermediario puro. Captura las interacciones y eventos de la vista, invoca de manera limpia los métodos de ConversorService y despacha los resultados a la vista, garantizando cero acoplamiento entre la capa de interfaz y la red.

3.4 Servidor Payara 6 y Jakarta REST (JAX-RS)

Payara Server 6 es un servidor de aplicaciones empresariales certificado para Jakarta EE 10. Provee un motor optimizado para Jakarta REST 3.1 (JAX-RS), permitiendo gestionar el ciclo de vida de los endpoints mediante anotaciones como @ApplicationPath, @Path, @GET, @POST, @Produces(MediaType.APPLICATION_JSON) y @Consumes(MediaType.APPLICATION_JSON).
Seguridad por Bearer Tokens y Filtros Perimetrales en Memoria (SIN BDD):
En entornos sin base de datos relacional, la autenticación se implementa mediante tokens criptográficos aleatorios generados tras la validación exitosa de credenciales en memoria (MONSTER / MONSTER9). Estos tokens se almacenan en estructuras concurrentes (ConcurrentHashMap) con expiración temporal configurada. Cada solicitud de conversión incluye el token en la cabecera HTTP estándar 'Authorization: Bearer <token>' o mediante parámetros JSON. Para permitir llamadas desde aplicaciones web de dominios externos o navegadores modernos, se implementa un filtro de ContainerResponseFilter (CorsFilter) que inyecta las cabeceras Access-Control-Allow-* requeridas.

4 DESARROLLO DEL APLICATIVO

El desarrollo del sistema conversor de unidades distribuidas comprende la construcción del servidor de servicios web RESTful y la implementación de cuatro clientes nativos heterogéneos (consola, escritorio, móvil y web), desarrollados íntegramente sobre la plataforma Java 17 y estructurados bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC).

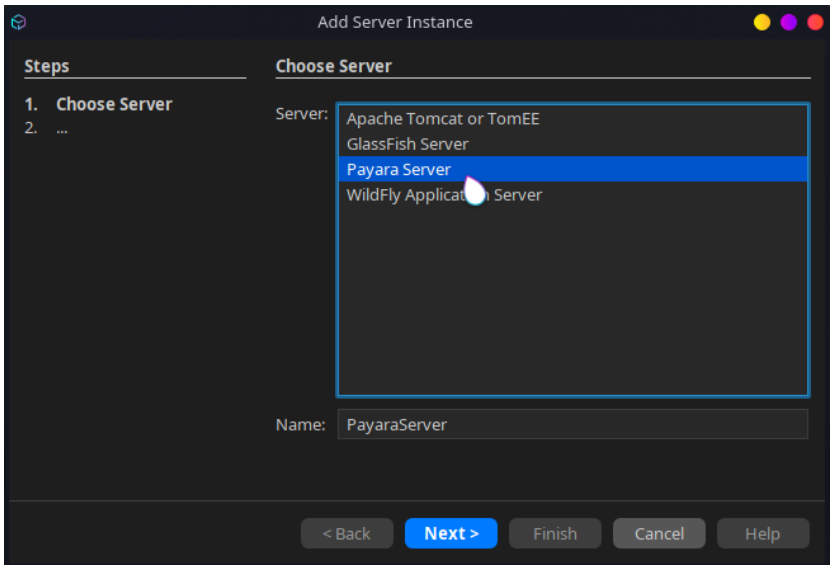
4.1 Servidor Web RESTful

4.1.1 Paso 1: Creación de la Instancia de Servidor Payara en el IDE

En el entorno de desarrollo integrado NetBeans, dentro del panel de administración de servicios (Services), se inicializa el asistente para el registro del servidor de aplicaciones seleccionando Payara Server.

Figura 1

Creación de Servidor Payara en NetBeans

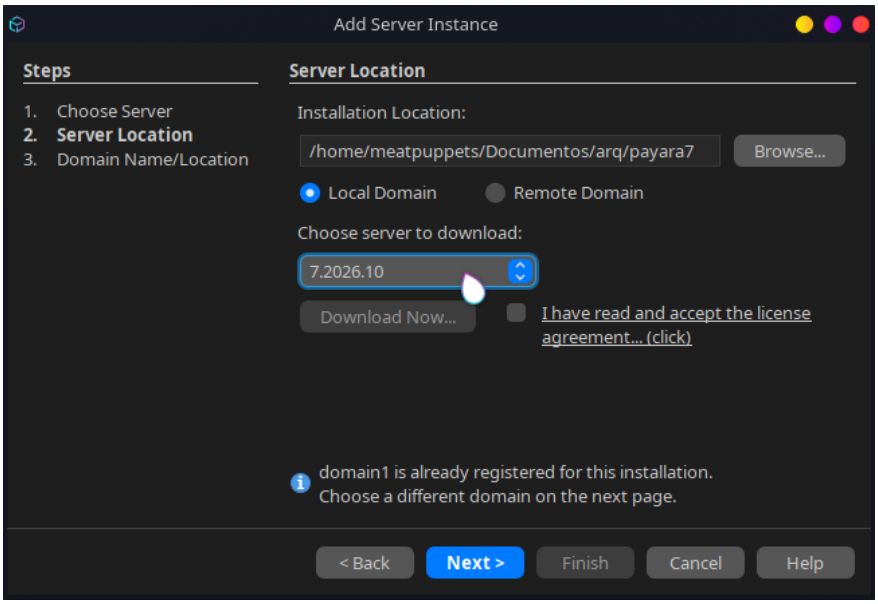


4.1.2 Paso 2: Selección de Versión y Directorio de Instalación

Se selecciona la versión estable de Payara Server Community compatible con Jakarta EE 10, se confirman los términos de licenciamiento de código abierto y se especifica la ruta del sistema de archivos donde se encuentran los binarios de instalación.

Figura 2

Configuración básica del servidor Payara

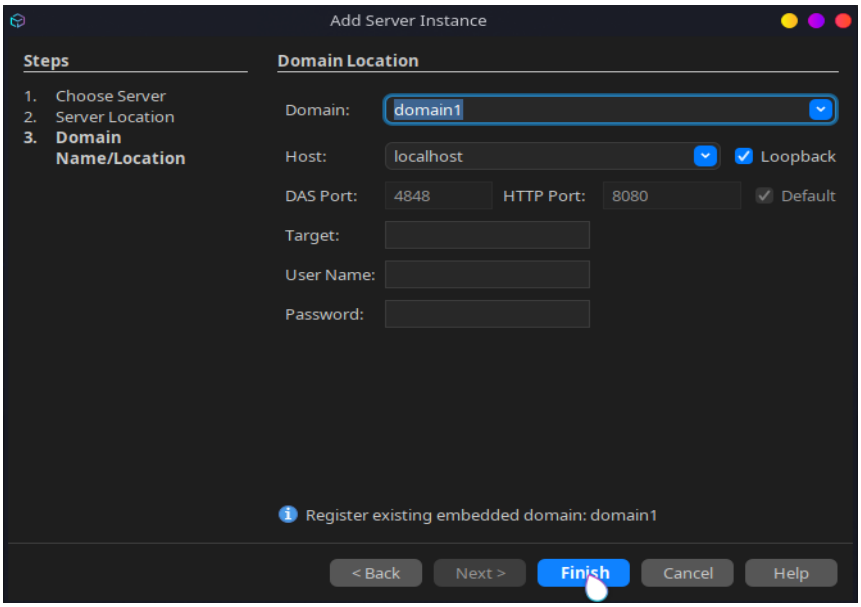


4.1.3 Paso 3: Configuración de Dominio y Puertos de Red

Se parametrizan las propiedades del dominio de ejecución (domain1), estableciendo el host de enlace local (localhost o interfaz de red configurada), el puerto del contenedor HTTP (8080) y el puerto de administración de la consola web (4848).

Figura 3

Configuración avanzada del servidor Payara

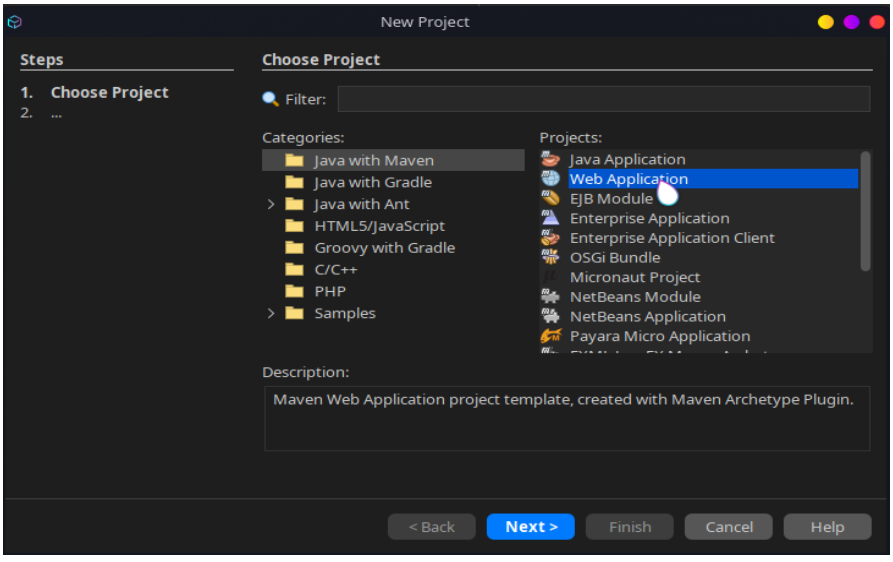


4.1.4 Paso 4: Creación del Proyecto Tipo Aplicación Web

Una vez activo el servidor de aplicaciones, se crea un nuevo proyecto en NetBeans clasificado como Java with Maven > Web Application, garantizando la gestión declarativa de dependencias y la compilación estandarizada mediante Maven.

Figura 4

Creación de Proyecto Tipo Aplicación Web REST

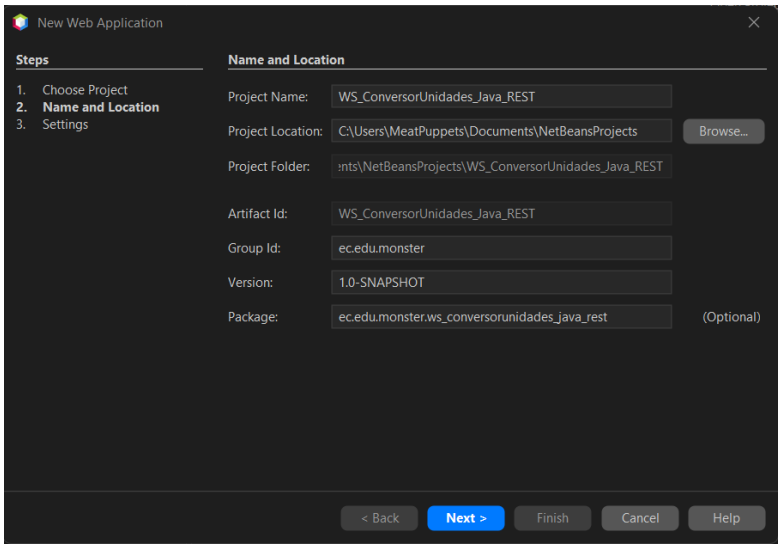


4.1.5 Paso 5: Parametrización de Metadatos del Proyecto

Se asigna el identificador de artefacto WS_ConversorUnidades_Java_REST, estableciendo el identificador de grupo (GroupId) bajo el estándar de nomenclatura inversa de dominio ec.edu.monster y la versión base 1.0-SNAPSHOT.

Figura 5

Parámetros del proyecto servidor Maven

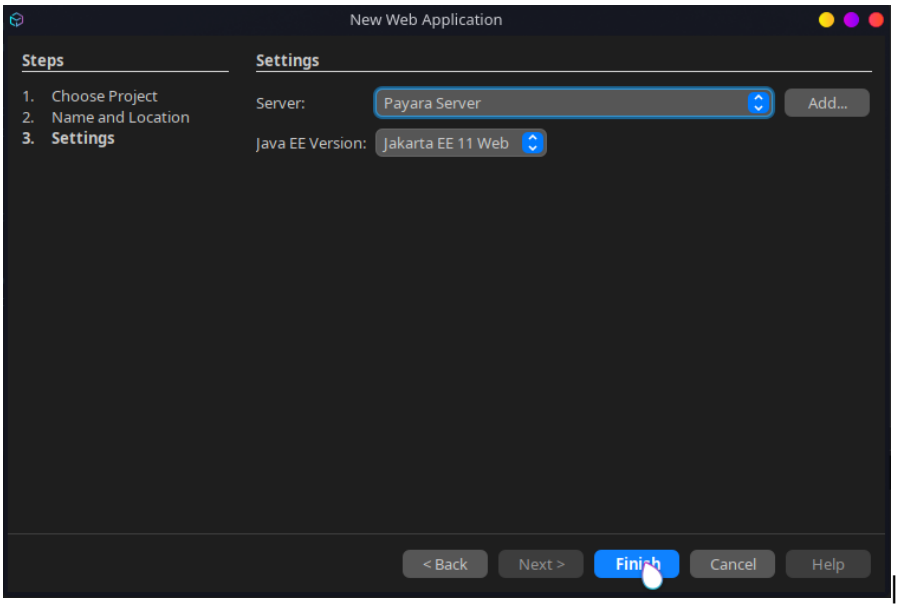


4.1.6 Paso 6: Asociación del Servidor y Plataforma Jakarta EE 10

Se asocia el proyecto recién creado al servidor Payara configurado en los pasos previos, definiendo Jakarta EE 10 como el conjunto de especificaciones empresariales de desarrollo para asegurar el soporte nativo de JAX-RS y CDI.

Figura 6

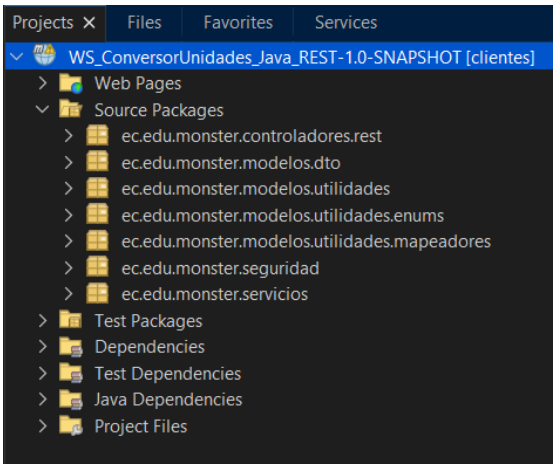
Selección de Servidor y perfil Jakarta EE 10



4.1.7 Paso 7: Estructuración de Paquetes del Servidor

Para garantizar una separación de responsabilidades estricta, se organiza el código fuente dentro del directorio src/main/java en la siguiente estructura modular: ec.edu.monster.controladores.rest, ec.edu.monster.modelos.dto, ec.edu.monster.modelos.utilidades, ec.edu.monster.seguridad y ec.edu.monster.servicios.

Figura 7
Estructura modular de paquetes del servidor REST



4.1.8 Paso 8: Configuración de la Aplicación REST (RestApplication.java)

Dentro del paquete ec.edu.monster.controladores.rest, se implementa la clase RestApplication que extiende de jakarta.ws.rs.core.Application decorada con la anotación @ApplicationPath('/api'). Esta clase activa el motor JAX-RS en Payara Server y define el prefijo base de todos los recursos y endpoints publicados:

Figura 8
Configuración de RestApplication (@ApplicationPath)

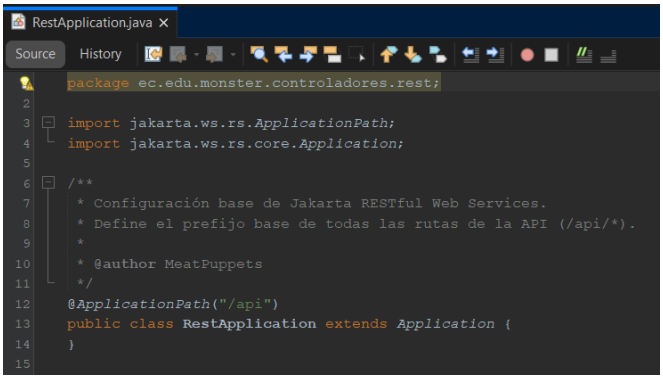


Tabla 1
Configuración de la aplicación REST (RestApplication.java)

```
package ec.edu.monster.controladores.rest;
import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

/**
 * ConfiguraciÃ³n base de Jakarta RESTful Web Services.
 * Define el prefijo base de todas las rutas de la API (/api/*).
 *
 * @author MeatPuppets
 */
@ApplicationPath("/api")
public class RestApplication extends Application {
}
```

4.1.9 Paso 9: Implementación del Filtro Global de CORS (CorsFilter.java)

Para permitir que clientes web basados en navegadores o clientes móviles consuman la API REST sin bloqueos de seguridad de origen cruzado, se implementa el proveedor CorsFilter que implementa ContainerResponseFilter:

Figura 9

Declaración de Endpoints RESTful con anotaciones JAX-RS

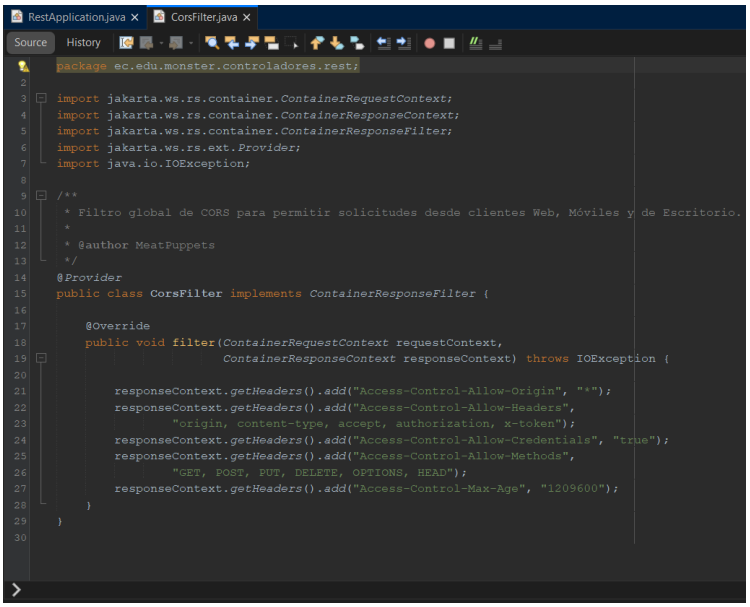


Tabla 2

Filtro global de CORS (CorsFilter.java)

```
package ec.edu.monster.controladores.rest;

import jakarta.ws.rs.container.ContainerRequestContext;
import jakarta.ws.rs.container.ContainerResponseContext;
import jakarta.ws.rs.container.ContainerResponseFilter;
import jakarta.ws.rs.ext.Provider;
import java.io.IOException;

/**
 * Filtro global de CORS para permitir solicitudes desde clientes Web, MÃ³viles y de Escritorio.
 *
 * @author MeatPuppets
 */
@Provider
public class CorsFilter implements ContainerResponseFilter {

    @Override
    public void filter(ContainerRequestContext requestContext,
                       ContainerResponseContext responseContext) throws IOException {

        responseContext.getHeaders().add("Access-Control-Allow-Origin", "*");
        responseContext.getHeaders().add("Access-Control-Allow-Headers",
                "origin, content-type, accept, authorization, x-token");
        responseContext.getHeaders().add("Access-Control-Allow-Credentials", "true");
```

```
        responseContext.getHeaders().add("Access-Control-Allow-Methods",
                "GET, POST, PUT, DELETE, OPTIONS, HEAD");
        responseContext.getHeaders().add("Access-Control-Max-Age", "1209600");
    }
}
```

4.1.10 Paso 10: Implementación del Endpoint Central de Compatibilidad (WSConversorUnidadesRest.java)

Se crea el controlador WSConversorUnidadesRest mapeado en @Path('/WSConversorUnidades'), exponiendo las operaciones login, cambiarContrasenia, convertirLongitud, convertirMasa y convertirTemperatura en formato JSON:

1. Declaración, inyecciones de servicio y método de login:

Tabla 3

Declaración del Endpoint REST y autenticación (WSConversorUnidadesRest.java)

```
package ec.edu.monster.controladores.rest;

import ec.edu.monster.modelos.dto.ConversionResponse;
import ec.edu.monster.modelos.dto.LoginResponse;
import ec.edu.monster.modelos.dto.RespuestaGeneral;
import ec.edu.monster.modelos.utilidades.ConversorLongitud;
import ec.edu.monster.modelos.utilidades.ConversorMasa;
import ec.edu.monster.modelos.utilidades.ConversorTemperatura;
import ec.edu.monster.modelos.utilidades.enums.UnidadLongitud;
import ec.edu.monster.modelos.utilidades.enums.UnidadMasa;
import ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura;
import ec.edu.monster.modelos.utilidades.mapeadores.UnidadMapper;
import ec.edu.monster.seguridad.AdministradorCredenciales;
import ec.edu.monster.seguridad.AdministradorToken;
import ec.edu.monster.servicios.ServicioConversor;

import jakarta.ws.rs.Consumes;
import jakarta.ws.rs.FormParam;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.POST;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.QueryParam;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;

/**
 * Endpoint de compatibilidad directa con los mÃ©todos del Web Service SOAP original.
 * Permite invocar las 5 operaciones principales con la misma nomenclatura:
 * - login
 * - cambiarContrasenia
 * - convertirLongitud
 * - convertirMasa
 * - convertirTemperatura
 *
 * @author MeatPuppets
```

```
 */
@Path("/WSConversorUnidades")
@Produces(MediaType.APPLICATION_JSON)
public class WSConversorUnidadesRest {

   private final ServicioConversor<UnidadLongitud> servicioLongitud =
         new ServicioConversor<>(new ConversorLongitud());
   private final ServicioConversor<UnidadMasa> servicioMasa =
         new ServicioConversor<>(new ConversorMasa());
   private final ServicioConversor<UnidadTemperatura> servicioTemperatura =
         new ServicioConversor<>(new ConversorTemperatura());

   @POST
   @Path("/login")
   @Consumes({MediaType.APPLICATION_JSON,
MediaType.APPLICATION_FORM_URLENCODED})
   public Response login(@QueryParam("usuario") String qUser,
                  @QueryParam("contrasenia") String qPass,
                  @FormParam("usuario") String fUser,
                  @FormParam("contrasenia") String fPass) {

      String u = (qUser != null) ? qUser : fUser;
      String p = (qPass != null) ? qPass : fPass;

      if (AdministradorCredenciales.validarUsuario(u, p)) {
         String token = AdministradorToken.generarToken();
         return Response.ok(new LoginResponse(true, token, u.toUpperCase(), "Login
correcto")).build();
      }
      return Response.status(Response.Status.UNAUTHORIZED)
            .entity(new LoginResponse(false, null, u, "Credenciales incorrectas")).build();
   }

   @POST
   @Path("/cambiarContrasenia")
   @Consumes({MediaType.APPLICATION_JSON,
MediaType.APPLICATION_FORM_URLENCODED})
   public Response cambiarContrasenia(@QueryParam("contraseniaActual") String qActual,
                           @QueryParam("contraseniaNueva") String qNueva,
                           @FormParam("contraseniaActual") String fActual,
                           @FormParam("contraseniaNueva") String fNueva) {

      String actual = (qActual != null) ? qActual : fActual;
      String nueva = (qNueva != null) ? qNueva : fNueva;

      try {
         AdministradorCredenciales.cambiarContrasenia(actual, nueva);
         return Response.ok(RespuestaGeneral.ok("Contrasenia actualizada
correctamente")).build();
      } catch (Exception e) {
```

```
            return Response.status(Response.Status.BAD_REQUEST)
                    .entity(RespuestaGeneral.error(e.getMessage())).build();
        }
    }
```

2. Métodos de conversión física (longitud, masa y temperatura) con soporte de GET y POST:

Tabla 4

Métodos de conversión expuestos en REST (WSConversorUnidadesRest.java)

```
    @GET
    @Path("/convertirLongitud")
    public Response convertirLongitudGet(@QueryParam("valor") double valor,
                                         @QueryParam("unidadInicial") String unidadInicial,
                                         @QueryParam("unidadFinal") String unidadFinal) {
        return convertirLongitud(valor, unidadInicial, unidadFinal);
    }

    @POST
    @Path("/convertirLongitud")
    @Consumes({MediaType.APPLICATION_JSON,
MediaType.APPLICATION_FORM_URLENCODED})
    public Response convertirLongitudPost(@FormParam("valor") double valor,
                                          @FormParam("unidadInicial") String unidadInicial,
                                          @FormParam("unidadFinal") String unidadFinal) {
        return convertirLongitud(valor, unidadInicial, unidadFinal);
    }

    private Response convertirLongitud(double valor, String unidadInicial, String unidadFinal) {
        try {
            UnidadLongitud origen = UnidadMapper.toLongitud(unidadInicial);
            UnidadLongitud destino = UnidadMapper.toLongitud(unidadFinal);
            double resultado = servicioLongitud.convertir(valor, origen, destino);
            return Response.ok(new ConversionResponse(true, "LONGITUD", valor, origen.name(),
resultado, destino.name(), "OK")).build();
        } catch (Exception e) {
            return
Response.status(Response.Status.BAD_REQUEST).entity(RespuestaGeneral.error(e.getMess
age())).build();
        }
    }

    @GET
    @Path("/convertirMasa")
    public Response convertirMasaGet(@QueryParam("valor") double valor,
                                     @QueryParam("unidadInicial") String unidadInicial,
                                     @QueryParam("unidadFinal") String unidadFinal) {
        return convertirMasa(valor, unidadInicial, unidadFinal);
    }

    @POST
```

```
    @Path("/convertirMasa")
    @Consumes({MediaType.APPLICATION_JSON,
MediaType.APPLICATION_FORM_URLENCODED})
    public Response convertirMasaPost(@FormParam("valor") double valor,
                                      @FormParam("unidadInicial") String unidadInicial,
                                      @FormParam("unidadFinal") String unidadFinal) {
        return convertirMasa(valor, unidadInicial, unidadFinal);
    }

    private Response convertirMasa(double valor, String unidadInicial, String unidadFinal) {
        try {
            UnidadMasa origen = UnidadMapper.toMasa(unidadInicial);
            UnidadMasa destino = UnidadMapper.toMasa(unidadFinal);
            double resultado = servicioMasa.convertir(valor, origen, destino);
            return Response.ok(new ConversionResponse(true, "MASA", valor, origen.name(),
resultado, destino.name(), "OK")).build();
        } catch (Exception e) {
            return
Response.status(Response.Status.BAD_REQUEST).entity(RespuestaGeneral.error(e.getMess
age())).build();
        }
    }

    @GET
    @Path("/convertirTemperatura")
    public Response convertirTemperaturaGet(@QueryParam("valor") double valor,
                                            @QueryParam("unidadInicial") String unidadInicial,
                                            @QueryParam("unidadFinal") String unidadFinal) {
        return convertirTemperatura(valor, unidadInicial, unidadFinal);
    }

    @POST
    @Path("/convertirTemperatura")
    @Consumes({MediaType.APPLICATION_JSON,
MediaType.APPLICATION_FORM_URLENCODED})
    public Response convertirTemperaturaPost(@FormParam("valor") double valor,
                                             @FormParam("unidadInicial") String unidadInicial,
                                             @FormParam("unidadFinal") String unidadFinal) {
        return convertirTemperatura(valor, unidadInicial, unidadFinal);
    }

    private Response convertirTemperatura(double valor, String unidadInicial, String
unidadFinal) {
        try {
            UnidadTemperatura origen = UnidadMapper.toTemperatura(unidadInicial);
            UnidadTemperatura destino = UnidadMapper.toTemperatura(unidadFinal);
            double resultado = servicioTemperatura.convertir(valor, origen, destino);
            return Response.ok(new ConversionResponse(true, "TEMPERATURA", valor,
origen.name(), resultado, destino.name(), "OK")).build();
        } catch (Exception e) {
```

```
        return
Response.status(Response.Status.BAD_REQUEST).entity(RespuestaGeneral.error(e.getMess
age())).build();
      }
   }
}
```

4.1.11 Paso 11: Implementación de Recursos REST Dedicados (/api/auth y /api/conversor)

Para una arquitectura orientada a recursos pura, se diseñan endpoints independientes para autenticación y para cálculo numérico:

AutenticacionResource maneja el login con cuerpo JSON (/api/auth/login), formularios (/api/auth/login-form), cambio de contraseña (/api/auth/cambiar-contrasenia) y verificación de vigencia de tokens:

Tabla 5

Recurso REST de Autenticación (AutenticacionResource.java)

```
package ec.edu.monster.controladores.rest;

import ec.edu.monster.modelos.dto.CambioContraseniaRequest;
import ec.edu.monster.modelos.dto.LoginRequest;
import ec.edu.monster.modelos.dto.LoginResponse;
import ec.edu.monster.modelos.dto.RespuestaGeneral;
import ec.edu.monster.seguridad.AdministradorCredenciales;
import ec.edu.monster.seguridad.AdministradorToken;

import jakarta.ws.rs.Consumes;
import jakarta.ws.rs.FormParam;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.HeaderParam;
import jakarta.ws.rs.POST;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.QueryParam;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;

/**
 * Endpoints REST para autenticaciÃ³n, gestiÃ³n de tokens y cambio de contraseÃ±as.
 *
 * @author MeatPuppets
 */
@Path("/auth")
@Produces(MediaType.APPLICATION_JSON)
public class AutenticacionResource {

   /**
    * Inicio de sesiÃ³n mediante cuerpo JSON.
    * POST /api/auth/login
    */
   @POST
```

```
    @Path("/login")
    @Consumes(MediaType.APPLICATION_JSON)
    public Response loginJson(LoginRequest request) {
        if (request == null) {
            return Response.status(Response.Status.BAD_REQUEST)
                    .entity(new LoginResponse(false, null, null, "Datos de inicio de sesiÃ³n no proporcionados"))
                    .build();
        }
        return procesarLogin(request.getUsuario(), request.getContrasenia());
    }

    /**
     * Inicio de sesiÃ³n mediante Formulario o Query Params.
     * POST /api/auth/login-form
     */
    @POST
    @Path("/login-form")
    @Consumes(MediaType.APPLICATION_FORM_URLENCODED)
    public Response loginForm(@FormParam("usuario") String usuario,
                              @FormParam("contrasenia") String contrasenia) {
        return procesarLogin(usuario, contrasenia);
    }

    private Response procesarLogin(String usuario, String contrasenia) {
        if (AdministradorCredenciales.validarUsuario(usuario, contrasenia)) {
            String token = AdministradorToken.generarToken();
            LoginResponse response = new LoginResponse(
                    true, token, usuario.trim().toUpperCase(), "AutenticaciÃ³n exitosa"
            );
            return Response.ok(response).build();
        }

        LoginResponse response = new LoginResponse(
                false, null, usuario, "Usuario o contraseÃ±a incorrectos"
        );
        return Response.status(Response.Status.UNAUTHORIZED).entity(response).build();
    }

    /**
     * Cambio de contraseÃ±a para el usuario autenticado.
     * POST /api/auth/cambiar-contrasenia
     */
    @POST
    @Path("/cambiar-contrasenia")
    @Consumes(MediaType.APPLICATION_JSON)
    public Response cambiarContrasenia(CambioContraseniaRequest request) {
        if (request == null) {
            return Response.status(Response.Status.BAD_REQUEST)
                    .entity(RespuestaGeneral.error("Datos requeridos no proporcionados"))
```

```
                .build();
        }

        try {
            AdministradorCredenciales.cambiarContrasenia(
                    request.getContraseniaActual(), request.getContraseniaNueva()
            );
            return Response.ok(RespuestaGeneral.ok("ContraseÃ±a actualizada
correctamente")).build();
        } catch (Exception e) {
            return Response.status(Response.Status.BAD_REQUEST)
                    .entity(RespuestaGeneral.error(e.getMessage()))
                    .build();
        }
    }

    /**
     * Valida si un token sigue vigente.
     * GET /api/auth/validar-token
     */
    @GET
    @Path("/validar-token")
    public Response validarToken(@HeaderParam("Authorization") String authHeader,
                                 @QueryParam("token") String tokenParam) {
        String token = extraerToken(authHeader, tokenParam);
        if (token != null && AdministradorToken.validarToken(token)) {
            return Response.ok(RespuestaGeneral.ok("Token vigente y vÃ¡lido")).build();
        }
        return Response.status(Response.Status.UNAUTHORIZED)
                .entity(RespuestaGeneral.error("Token invÃ¡lido o expirado"))
                .build();
    }

    /**
     * Cierra sesiÃ³n invalidando el token.
     * POST /api/auth/logout
     */
    @POST
    @Path("/logout")
    public Response logout(@HeaderParam("Authorization") String authHeader,
                           @QueryParam("token") String tokenParam) {
        String token = extraerToken(authHeader, tokenParam);
        if (token != null) {
            AdministradorToken.invalidarToken(token);
        }
        return Response.ok(RespuestaGeneral.ok("Sesión cerrada correctamente")).build();
    }

    private String extraerToken(String authHeader, String tokenParam) {
        if (tokenParam != null && !tokenParam.trim().isEmpty()) {
```

```
            return tokenParam.trim();
        }
        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            return authHeader.substring(7).trim();
        }
        return authHeader;
    }
}
```

ConversorResource expone endpoints especializados para longitud (/api/conversor/longitud), masa (/api/conversor/masa) y temperatura (/api/conversor/temperatura):

Tabla 6

Recurso REST de Conversión (ConversorResource.java)

```
package ec.edu.monster.controladores.rest;

import ec.edu.monster.modelos.dto.ConversionRequest;
import ec.edu.monster.modelos.dto.ConversionResponse;
import ec.edu.monster.modelos.dto.RespuestaGeneral;
import ec.edu.monster.modelos.utilidades.ConversorLongitud;
import ec.edu.monster.modelos.utilidades.ConversorMasa;
import ec.edu.monster.modelos.utilidades.ConversorTemperatura;
import ec.edu.monster.modelos.utilidades.enums.UnidadLongitud;
import ec.edu.monster.modelos.utilidades.enums.UnidadMasa;
import ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura;
import ec.edu.monster.modelos.utilidades.mapeadores.UnidadMapper;
import ec.edu.monster.servicios.ServicioConversor;

import jakarta.ws.rs.Consumes;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.POST;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.QueryParam;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
import java.util.Arrays;
import java.util.Map;

/**
 * Endpoints RESTful para la conversiÃ³n de unidades (Longitud, Masa y Temperatura).
 * Soporta solicitudes POST con cuerpo JSON y solicitudes GET con Query Parameters.
 *
 * @author MeatPuppets
 */
@Path("/conversor")
@Produces(MediaType.APPLICATION_JSON)
public class ConversorResource {

    private final ServicioConversor<UnidadLongitud> servicioLongitud =
            new ServicioConversor<>(new ConversorLongitud());
```

```
    private final ServicioConversor<UnidadMasa> servicioMasa =
            new ServicioConversor<>(new ConversorMasa());
    private final ServicioConversor<UnidadTemperatura> servicioTemperatura =
            new ServicioConversor<>(new ConversorTemperatura());

    //
============================================================================
    // CONVERSIÃ“N DE LONGITUD
    //
============================================================================

    @POST
    @Path("/longitud")
    @Consumes(MediaType.APPLICATION_JSON)
    public Response convertirLongitudPost(ConversionRequest request) {
        if (request == null) {
            return Response.status(Response.Status.BAD_REQUEST)
                    .entity(RespuestaGeneral.error("Cuerpo de solicitud requerido"))
                    .build();
        }
        return ejecutarConversionLongitud(request.getValor(), request.getUnidadInicial(),
request.getUnidadFinal());
    }

    @GET
    @Path("/longitud")
    public Response convertirLongitudGet(@QueryParam("valor") double valor,
                                         @QueryParam("unidadInicial") String unidadInicial,
                                         @QueryParam("unidadFinal") String unidadFinal) {
        return ejecutarConversionLongitud(valor, unidadInicial, unidadFinal);
    }

    private Response ejecutarConversionLongitud(double valor, String inicial, String finalU) {
        try {
            UnidadLongitud origen = UnidadMapper.toLongitud(inicial);
            UnidadLongitud destino = UnidadMapper.toLongitud(finalU);
            double resultado = servicioLongitud.convertir(valor, origen, destino);

            ConversionResponse resp = new ConversionResponse(
                    true, "LONGITUD", valor, origen.name(), resultado, destino.name(), "ConversiÃ³n
exitosa"
            );
            return Response.ok(resp).build();
        } catch (IllegalArgumentException e) {
            return Response.status(Response.Status.BAD_REQUEST)
                    .entity(RespuestaGeneral.error(e.getMessage()))
                    .build();
        } catch (Exception e) {
            return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
```

```
            .entity(RespuestaGeneral.error("Error interno en la conversiÃ³n: " +
e.getMessage()))
            .build();
      }
   }

   //
==========================================================================
   // CONVERSIÃ“N DE MASA
   //
==========================================================================

   @POST
   @Path("/masa")
   @Consumes(MediaType.APPLICATION_JSON)
   public Response convertirMasaPost(ConversionRequest request) {
      if (request == null) {
         return Response.status(Response.Status.BAD_REQUEST)
               .entity(RespuestaGeneral.error("Cuerpo de solicitud requerido"))
               .build();
      }
      return ejecutarConversionMasa(request.getValor(), request.getUnidadInicial(),
request.getUnidadFinal());
   }

   @GET
   @Path("/masa")
   public Response convertirMasaGet(@QueryParam("valor") double valor,
                           @QueryParam("unidadInicial") String unidadInicial,
                           @QueryParam("unidadFinal") String unidadFinal) {
      return ejecutarConversionMasa(valor, unidadInicial, unidadFinal);
   }

   private Response ejecutarConversionMasa(double valor, String inicial, String finalU) {
      try {
         UnidadMasa origen = UnidadMapper.toMasa(inicial);
         UnidadMasa destino = UnidadMapper.toMasa(finalU);
         double resultado = servicioMasa.convertir(valor, origen, destino);

         ConversionResponse resp = new ConversionResponse(
               true, "MASA", valor, origen.name(), resultado, destino.name(), "ConversiÃ³n
exitosa"
         );
         return Response.ok(resp).build();
      } catch (IllegalArgumentException e) {
         return Response.status(Response.Status.BAD_REQUEST)
               .entity(RespuestaGeneral.error(e.getMessage()))
               .build();
      } catch (Exception e) {
         return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
```

```
                .entity(RespuestaGeneral.error("Error interno en la conversiÃ³n: " +
e.getMessage()))
                .build();
        }
    }

    //
==========================================================================
    // CONVERSIÃ“N DE TEMPERATURA
    //
==========================================================================

    @POST
    @Path("/temperatura")
    @Consumes(MediaType.APPLICATION_JSON)
    public Response convertirTemperaturaPost(ConversionRequest request) {
        if (request == null) {
            return Response.status(Response.Status.BAD_REQUEST)
                .entity(RespuestaGeneral.error("Cuerpo de solicitud requerido"))
                .build();
        }
        return ejecutarConversionTemperatura(request.getValor(), request.getUnidadInicial(),
request.getUnidadFinal());
    }

    @GET
    @Path("/temperatura")
    public Response convertirTemperaturaGet(@QueryParam("valor") double valor,
                                            @QueryParam("unidadInicial") String unidadInicial,
                                            @QueryParam("unidadFinal") String unidadFinal) {
        return ejecutarConversionTemperatura(valor, unidadInicial, unidadFinal);
    }

    private Response ejecutarConversionTemperatura(double valor, String inicial, String finalU)
{
        try {
            UnidadTemperatura origen = UnidadMapper.toTemperatura(inicial);
            UnidadTemperatura destino = UnidadMapper.toTemperatura(finalU);
            double resultado = servicioTemperatura.convertir(valor, origen, destino);

            ConversionResponse resp = new ConversionResponse(
                true, "TEMPERATURA", valor, origen.name(), resultado, destino.name(),
"ConversiÃ³n exitosa"
            );
            return Response.ok(resp).build();
        } catch (IllegalArgumentException e) {
            return Response.status(Response.Status.BAD_REQUEST)
                .entity(RespuestaGeneral.error(e.getMessage()))
                .build();
        } catch (Exception e) {
```

```
            return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
                    .entity(RespuestaGeneral.error("Error interno en la conversiÃ³n: " +
e.getMessage()))
                    .build();
        }
    }

    //
================================================================================
    // CATÃ•LOGO DE UNIDADES DISPONIBLES
    //
================================================================================

    @GET
    @Path("/unidades")
    public Response catalogoUnidades() {
        Map<String, Object> catalogo = Map.of(
                "longitud", Arrays.stream(UnidadLongitud.values()).map(Enum::name).toArray(),
                "masa", Arrays.stream(UnidadMasa.values()).map(Enum::name).toArray(),
                "temperatura",
Arrays.stream(UnidadTemperatura.values()).map(Enum::name).toArray()
        );
        return Response.ok(RespuestaGeneral.ok("CatÃ¡logo de unidades disponibles",
catalogo)).build();
    }
}
```

4.1.12 Paso 12: Implementación de la Capa de Dominio y Conversión

La lógica matemática y física se desacopla mediante la interfaz genérica IConversor<T>, sus implementaciones concretas para longitud, masa y temperatura, y el mapeador UnidadMapper:

Tabla 7

Interfaz Conversor (IConversor.java)

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Interface.java to edit this
template
 */
package ec.edu.monster.modelos.utilidades;

/**
 *
 * @author MeatPuppets
 */
public interface IConversor<T> {
    public double convertir(double valor, T unidadInicial, T unidadFinal);

}
```

Tabla 8

Conversor Longitud (ConversorLongitud.java)

```
package ec.edu.monster.modelos.utilidades;

import ec.edu.monster.modelos.utilidades.enums.UnidadLongitud;
import java.util.Map;

public class ConversorLongitud implements IConversor<UnidadLongitud> {

   private final Map<UnidadLongitud, Double> factores = Map.of(
      UnidadLongitud.MILIMETRO, 0.001,
      UnidadLongitud.CENTIMETRO, 0.01,
      UnidadLongitud.METRO, 1.0,
      UnidadLongitud.KILOMETRO, 1000.0,
      UnidadLongitud.YARDA, 0.9144
   );

   public ConversorLongitud() {}

   @Override
   public double convertir(double valor, UnidadLongitud unidadOrigen, UnidadLongitud
unidadFinal) {

      if (unidadOrigen == null || unidadFinal == null) {
         throw new IllegalArgumentException("Las unidades no pueden ser null");
      }
      if (!factores.containsKey(unidadOrigen) || !factores.containsKey(unidadFinal)) {
         throw new IllegalArgumentException("Unidad no soportada");
      }

      double valorEnBase = valor * factores.get(unidadOrigen);

      return valorEnBase / factores.get(unidadFinal);
   }
}
```

Tabla 9

Conversor Masa (ConversorMasa.java)

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.monster.modelos.utilidades;
import ec.edu.monster.modelos.utilidades.enums.UnidadMasa;
import java.util.Map;


/**
```

```
 *
 * @author MeatPuppets
 */
public class ConversorMasa implements IConversor<UnidadMasa> {

   private final Map<UnidadMasa, Double> factores = Map.of(
      UnidadMasa.MILIGRAMO, 0.001,
      UnidadMasa.GRAMO, 1.0,
      UnidadMasa.KILOGRAMO, 1000.0,
      UnidadMasa.TONELADA, 1_000_000.0,
      UnidadMasa.ONZA, 28.3495
   );

   public ConversorMasa() {}

   @Override
   public double convertir(double valor, UnidadMasa unidadOrigen, UnidadMasa unidadFinal)
{

      if (unidadOrigen == null || unidadFinal == null) {
         throw new IllegalArgumentException("Las unidades no pueden ser null");
      }
      if (!factores.containsKey(unidadOrigen) || !factores.containsKey(unidadFinal)) {
         throw new IllegalArgumentException("Unidad no soportada");
      }

      double valorEnBase = valor * factores.get(unidadOrigen);

      return valorEnBase / factores.get(unidadFinal);
   }
}
```

Tabla 10

Conversor Temperatura (ConversorTemperatura.java)

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.monster.modelos.utilidades;

import ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura;
import static ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura.CELSIUS;
import static ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura.FAHRENHEIT;
import static ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura.KELVIN;
import static ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura.RANKINE;


/**
 *
```

```
 * @author MeatPuppets
 */
public class ConversorTemperatura implements IConversor<UnidadTemperatura> {

   public ConversorTemperatura() {}

   @Override
   public double convertir(double valor, UnidadTemperatura origen, UnidadTemperatura
destino) {

      if (origen == null || destino == null) {
         throw new IllegalArgumentException("Las unidades no pueden ser null");
      }
      if (origen == destino) {
         return valor;
      }

      return switch (origen) {

         case CELSIUS -> switch (destino) {
            case FAHRENHEIT -> (valor * 9 / 5) + 32;
            case KELVIN -> valor + 273.15;
            case RANKINE -> (valor + 273.15) * 9 / 5;
            default -> throw new IllegalArgumentException("ConversiÃ³n no soportada");
         };

         case FAHRENHEIT -> switch (destino) {
            case CELSIUS -> (valor - 32) * 5 / 9;
            case KELVIN -> (valor - 32) * 5 / 9 + 273.15;
            case RANKINE -> valor + 459.67;
            default -> throw new IllegalArgumentException("ConversiÃ³n no soportada");
         };

         case KELVIN -> switch (destino) {
            case CELSIUS -> valor - 273.15;
            case FAHRENHEIT -> (valor - 273.15) * 9 / 5 + 32;
            case RANKINE -> valor * 9 / 5;
            default -> throw new IllegalArgumentException("ConversiÃ³n no soportada");
         };

         case RANKINE -> switch (destino) {
            case CELSIUS -> (valor - 491.67) * 5 / 9;
            case FAHRENHEIT -> valor - 459.67;
            case KELVIN -> valor * 5 / 9;
            default -> throw new IllegalArgumentException("ConversiÃ³n no soportada");
         };
      };
   }
}
```

Tabla 11

Servicio de Dominio Conversor (ServicioConversor.java)

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.monster.servicios;
import ec.edu.monster.modelos.utilidades.IConversor;
/**
 *
 * @author MeatPuppets
 */
public class ServicioConversor<T> {

    private final IConversor<T> conversor;

    public ServicioConversor(IConversor<T> conversor) {
        this.conversor = conversor;
    }

    public double convertir(double valor, T origen, T destino) {
        if (conversor == null) {
            throw new IllegalStateException("No se ha inyectado un conversor");
        }
        return conversor.convertir(valor, origen, destino);
    }
}
```

Tabla 12

Mapeador de Unidades (UnidadMapper.java)

```
package ec.edu.monster.modelos.utilidades.mapeadores;

import ec.edu.monster.modelos.utilidades.enums.UnidadLongitud;
import ec.edu.monster.modelos.utilidades.enums.UnidadMasa;
import ec.edu.monster.modelos.utilidades.enums.UnidadTemperatura;
import java.text.Normalizer;

/**
 * Mapeador para convertir cadenas de texto a los respectivos enumeradores de unidad.
 * Soporta nombres en mayÃºsculas/minÃºsculas, con/sin tildes, plurales y abreviaturas comunes.
 *
 * @author MeatPuppets
 */
public class UnidadMapper {

    private UnidadMapper() {
```

```
    // Constructor privado para clase utilitaria
}

private static String normalizar(String texto) {
    if (texto == null || texto.trim().isEmpty()) {
        throw new IllegalArgumentException("La unidad no puede estar vacÃ-a.");
    }
    String limpio = Normalizer.normalize(texto.trim(), Normalizer.Form.NFD)
            .replaceAll("\\p{M}", "")
            .toUpperCase();
    return limpio;
}

/**
 * Mapea una cadena de texto a UnidadLongitud.
 *
 * @param valor Cadena representativa de la unidad.
 * @return UnidadLongitud correspondiente.
 */
public static UnidadLongitud toLongitud(String valor) {
    String u = normalizar(valor);
    return switch (u) {
        case "MILIMETRO", "MILIMETROS", "MM" -> UnidadLongitud.MILIMETRO;
        case "CENTIMETRO", "CENTIMETROS", "CM" -> UnidadLongitud.CENTIMETRO;
        case "METRO", "METROS", "M" -> UnidadLongitud.METRO;
        case "KILOMETRO", "KILOMETROS", "KM" -> UnidadLongitud.KILOMETRO;
        case "YARDA", "YARDAS", "YD" -> UnidadLongitud.YARDA;
        default -> {
            try {
                yield UnidadLongitud.valueOf(u);
            } catch (IllegalArgumentException e) {
                throw new IllegalArgumentException("Unidad de longitud no soportada: " + valor);
            }
        }
    };
}

/**
 * Mapea una cadena de texto a UnidadMasa.
 *
 * @param valor Cadena representativa de la unidad.
 * @return UnidadMasa correspondiente.
 */
public static UnidadMasa toMasa(String valor) {
    String u = normalizar(valor);
    return switch (u) {
        case "MILIGRAMO", "MILIGRAMOS", "MG" -> UnidadMasa.MILIGRAMO;
        case "GRAMO", "GRAMOS", "G" -> UnidadMasa.GRAMO;
        case "KILOGRAMO", "KILOGRAMOS", "KG" -> UnidadMasa.KILOGRAMO;
        case "TONELADA", "TONELADAS", "T" -> UnidadMasa.TONELADA;
```

```
            case "ONZA", "ONZAS", "OZ" -> UnidadMasa.ONZA;
            default -> {
                try {
                    yield UnidadMasa.valueOf(u);
                } catch (IllegalArgumentException e) {
                    throw new IllegalArgumentException("Unidad de masa no soportada: " + valor);
                }
            }
        };
    }

    /**
     * Mapea una cadena de texto a UnidadTemperatura.
     *
     * @param valor Cadena representativa de la unidad.
     * @return UnidadTemperatura correspondiente.
     */
    public static UnidadTemperatura toTemperatura(String valor) {
        String u = normalizar(valor);
        return switch (u) {
            case "CELSIUS", "CENTIGRADO", "CENTIGRADOS", "C", "GRADOS CELSIUS" ->
UnidadTemperatura.CELSIUS;
            case "FAHRENHEIT", "F", "GRADOS FAHRENHEIT" -> UnidadTemperatura.FAHRENHEIT;
            case "KELVIN", "K", "GRADOS KELVIN" -> UnidadTemperatura.KELVIN;
            case "RANKINE", "R", "GRADOS RANKINE" -> UnidadTemperatura.RANKINE;
            default -> {
                try {
                    yield UnidadTemperatura.valueOf(u);
                } catch (IllegalArgumentException e) {
                    throw new IllegalArgumentException("Unidad de temperatura no soportada: " +
valor);
                }
            }
        };
    }
}
```

4.1.13 Paso 13: Implementación del Manejador de Seguridad y Sesión en Memoria (SIN BDD)

Para satisfacer la arquitectura sin base de datos, AdministradorCredenciales gestiona usuarios concurrentes con credencial inicial MONSTER / MONSTER9, y AdministradorToken emite tokens criptográficos con tiempo de expiración:

Figura 10
Implementación del filtro de CORS y seguridad en memoria

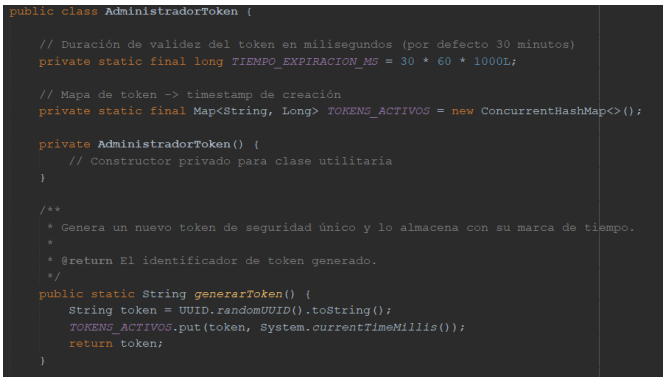


Tabla 13

Administrador de Credenciales en Memoria (AdministradorCredenciales.java)

```
package ec.edu.monster.seguridad;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Administrador de credenciales en memoria (sin base de datos).
 * Gestiona la autenticaciÃ³n y cambio de contraseÃ±as de los usuarios.
 *
 * @author MeatPuppets
 */
public class AdministradorCredenciales {

   private static final Map<String, String> USUARIOS = new ConcurrentHashMap<>();

   // Credencial Ãºnica permitida en el sistema
   public static final String USUARIO_DEFAULT = "MONSTER";
   public static final String CONTRASENIA_DEFAULT = "MONSTER9";

   static {
      reiniciar();
   }

   private AdministradorCredenciales() {
      // Constructor privado para clase utilitaria
   }

   /**
    * Restablece las credenciales a la Ãºnica credencial permitida:
    * USUARIO: MONSTER
    * PASSWORD: MONSTER9
    */
   public static void reiniciar() {
      USUARIOS.clear();
```

```
        USUARIOS.put(USUARIO_DEFAULT.toLowerCase(), CONTRASENIA_DEFAULT);
    }

    /**
     * Valida si el usuario y la contraseÃ±a coinciden.
     *
     * @param usuario     Nombre de usuario.
     * @param contrasenia ContraseÃ±a ingresada.
     * @return true si las credenciales son vÃ¡lidas, false en caso contrario.
     */
    public static boolean validarUsuario(String usuario, String contrasenia) {
        if (usuario == null || contrasenia == null) {
            return false;
        }
        String claveAlmacenada = USUARIOS.get(usuario.trim().toLowerCase());
        return claveAlmacenada != null && claveAlmacenada.equals(contrasenia);
    }

    /**
     * Cambia la contraseÃ±a buscando el usuario cuya contraseÃ±a actual coincide con
     * la provista.
     * Si no se encuentra coincidencia, lanza RuntimeException.
     *
     * @param contraseniaActual ContraseÃ±a actual del usuario.
     * @param contraseniaNueva  Nueva contraseÃ±a a establecer.
     */
    public static void cambiarContrasenia(String contraseniaActual, String contraseniaNueva) {
        if (contraseniaActual == null || contraseniaNueva == null ||
contraseniaNueva.trim().isEmpty()) {
            throw new RuntimeException("Las contraseÃ±as no pueden ser nulas ni vacÃ-as.");
        }

        boolean actualizada = false;
        for (Map.Entry<String, String> entry : USUARIOS.entrySet()) {
            if (entry.getValue().equals(contraseniaActual)) {
                USUARIOS.put(entry.getKey(), contraseniaNueva);
                actualizada = true;
                break;
            }
        }

        if (!actualizada) {
            throw new RuntimeException("La contraseÃ±a actual es incorrecta.");
        }
    }

    /**
     * Cambia la contraseÃ±a para un usuario especÃ-fico.
     *
     * @param usuario         Nombre de usuario.
```

```
    * @param contraseniaActual ContraseÃ±a actual.
    * @param contraseniaNueva  Nueva contraseÃ±a.
    */
   public static void cambiarContrasenia(String usuario, String contraseniaActual, String
contraseniaNueva) {
      if (usuario == null || contraseniaActual == null || contraseniaNueva == null
            || contraseniaNueva.trim().isEmpty()) {
         throw new RuntimeException("Datos invÃ¡lidos para cambio de contraseÃ±a.");
      }
      String userKey = usuario.trim().toLowerCase();
      String claveActual = USUARIOS.get(userKey);
      if (claveActual == null || !claveActual.equals(contraseniaActual)) {
         throw new RuntimeException("Credenciales incorrectas para el usuario: " + usuario);
      }
      USUARIOS.put(userKey, contraseniaNueva);
   }

   /**
    * Registra o actualiza un usuario en memoria.
    *
    * @param usuario     Nombre de usuario.
    * @param contrasenia ContraseÃ±a del usuario.
    */
   public static void registrarUsuario(String usuario, String contrasenia) {
      if (usuario != null && contrasenia != null && !usuario.isBlank() && !contrasenia.isBlank())
{
         USUARIOS.put(usuario.trim().toLowerCase(), contrasenia);
      }
   }
}
```

Tabla 14

Administrador de Tokens de Sesión (AdministradorToken.java)

```
package ec.edu.monster.seguridad;

import java.util.Map;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Administrador de tokens de seguridad en memoria.
 * Genera y valida tokens con tiempo de expiraciÃ³n configurable.
 *
 * @author MeatPuppets
 */
public class AdministradorToken {

   // DuraciÃ³n de validez del token en milisegundos (por defecto 30 minutos)
   private static final long TIEMPO_EXPIRACION_MS = 30 * 60 * 1000L;
```

```
// Mapa de token -> timestamp de creaciÃ³n
private static final Map<String, Long> TOKENS_ACTIVOS = new ConcurrentHashMap<>();

private AdministradorToken() {
   // Constructor privado para clase utilitaria
}

/**
 * Genera un nuevo token de seguridad Ãºnico y lo almacena con su marca de tiempo.
 *
 * @return El identificador de token generado.
 */
public static String generarToken() {
   String token = UUID.randomUUID().toString();
   TOKENS_ACTIVOS.put(token, System.currentTimeMillis());
   return token;
}

/**
 * Valida si un token existe y no ha expirado.
 *
 * @param token El token a validar.
 * @return true si el token es vÃ¡lido y estÃ¡ vigente, false en caso contrario.
 */
public static boolean validarToken(String token) {
   if (token == null || token.trim().isEmpty()) {
      return false;
   }

   Long timestampCreacion = TOKENS_ACTIVOS.get(token.trim());
   if (timestampCreacion == null) {
      return false;
   }

   // Verificar si expirÃ³
   if (System.currentTimeMillis() - timestampCreacion > TIEMPO_EXPIRACION_MS) {
      TOKENS_ACTIVOS.remove(token.trim());
      return false;
   }

   return true;
}

/**
 * Invalida (elimina) un token de la sesiÃ³n activa.
 *
 * @param token El token a invalidar.
 */
public static void invalidarToken(String token) {
   if (token != null) {
```

```
            TOKENS_ACTIVOS.remove(token.trim());
        }
    }

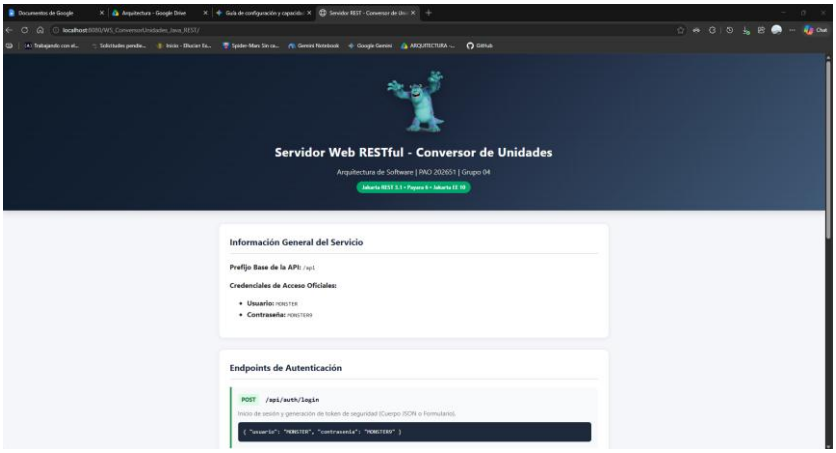
    /**
     * Limpia todos los tokens almacenados.
     */
    public static void limpiar() {
        TOKENS_ACTIVOS.clear();
    }
}
```

4.1.14 Paso 14: Despliegue en Servidor y Comprobación de Endpoints REST

Se compila la aplicación y se despliega sobre Payara Server 6. Se verifica la correcta exposición de los endpoints REST accediendo a la URL del servicio en el navegador o mediante cURL: http://localhost:8080/WS_ConversorUnidades_Java_REST/api/WSConversorUnidades/convertirLongitud?valor=100&unidadInicial=METRO&unidadFinal=CENTIMETRO, obteniendo una respuesta JSON estructurada.

Figura 11

Endpoint RESTful y recursos JSON publicados en Payara Server



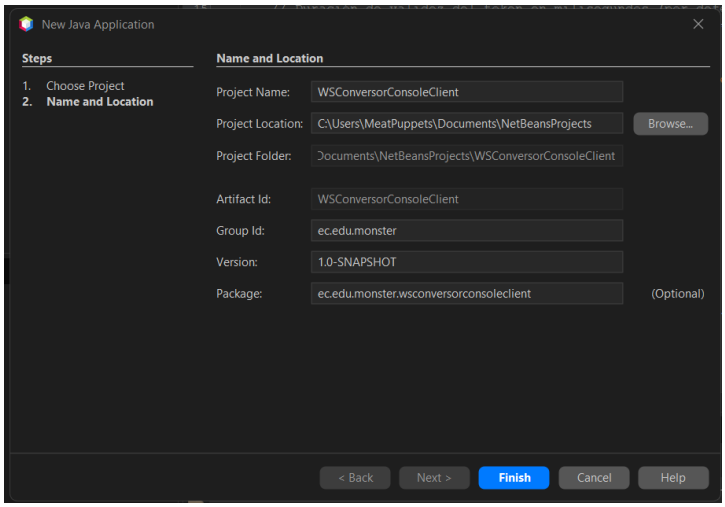
4.2 Cliente Consola

4.2.1 Paso 1: Creación del Proyecto Maven en NetBeans

En el entorno NetBeans IDE, se crea un proyecto Java with Maven > Java Application denominado WSConversorConsoleClient bajo el GroupId ec.edu.monster.

Figura 12

Creación del proyecto Maven para el Cliente Consola

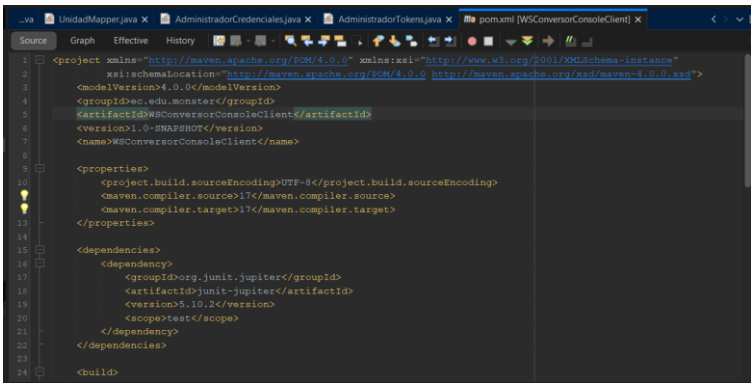


4.2.2 Paso 2: Configuración del Cliente HTTP y Dependencias Maven

A diferencia del cliente SOAP que requería wsimport y plugins de código proxy XML, el cliente RESTful utiliza Java puro nativo con HttpURLConnection y parseo ligero JSON, manteniendo dependencias mínimas en pom.xml.

Figura 13

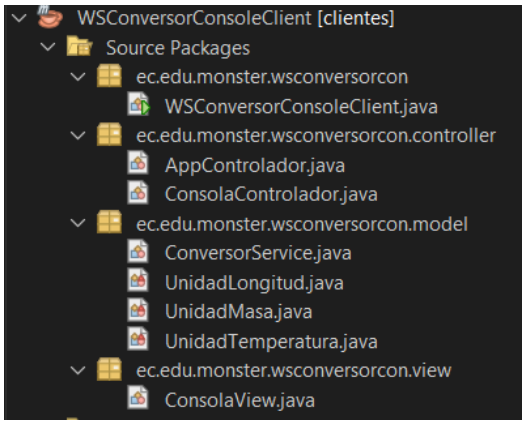
Configuración del pom.xml y dependencias del Cliente Consola



4.2.3 Paso 3: Estructuración Modular de Paquetes (MVC Estricto)

Se organiza el código fuente bajo la arquitectura estricta de 3 paquetes: ec.edu.monster.wsconversorcon.model, ec.edu.monster.wsconversorcon.view y ec.edu.monster.wsconversorcon.controller.

Figura 14
Estructura de paquetes MVC del Cliente Consola en NetBeans



4.2.4 Paso 4: Implementación de la Capa de Modelo (ConversorService.java)

La clase ConversorService implementa la comunicación HTTP RESTful, despachando peticiones POST para autenticación, capturando el Bearer token y realizando peticiones GET/POST con cabecera Authorization para las conversiones:

Tabla 15

Modelo de comunicación HTTP REST en Cliente Consola (ConversorService.java)

```
package ec.edu.monster.wsconversorcon.model;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ConversorService {

   private static final String BASE_URL =
"http://localhost:8080/WS_ConversorUnidades_Java_REST/api";
   private static String tokenSesion;

   public static String getTokenSesion() {
      return tokenSesion;
   }

   public static void setTokenSesion(String token) {
      tokenSesion = token;
   }

   public String login(String usuario, String contrasenia) {
      try {
```

```
        String jsonInput = String.format("{\"usuario\":\"%s\",\"contrasenia\":\"%s\"}",
                escapeJson(usuario), escapeJson(contrasenia));

        String respuesta = postJson(BASE_URL + "/auth/login", jsonInput);
        String token = extraerCampoJson(respuesta, "token");
        if (token != null && !token.isBlank()) {
           tokenSesion = token;
           return token;
        }
        return null;
     } catch (Exception e) {
        throw new RuntimeException("Error al iniciar sesiÃ³n: " + e.getMessage(), e);
     }
   }

   public double convertirLongitud(double valor, String unidadInicial, String unidadFinal) {
     return ejecutarConversion("longitud", valor, unidadInicial, unidadFinal);
   }

   public double convertirMasa(double valor, String unidadInicial, String unidadFinal) {
     return ejecutarConversion("masa", valor, unidadInicial, unidadFinal);
   }

   public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal)
{
     return ejecutarConversion("temperatura", valor, unidadInicial, unidadFinal);
   }

   public String cambiarContrasenia(String actual, String nueva) {
     try {
        String                                          jsonInput                                          =
String.format("{\"contraseniaActual\":\"%s\",\"contraseniaNueva\":\"%s\"}",
                escapeJson(actual), escapeJson(nueva));
        return postJson(BASE_URL + "/auth/cambiar-contrasenia", jsonInput);
     } catch (Exception e) {
        throw new RuntimeException("Error al cambiar contraseÃ±a: " + e.getMessage(), e);
     }
   }

   private double ejecutarConversion(String tipo, double valor, String inicial, String finalU) {
     try {
        String                                          urlStr                                          =
String.format("%s/conversor/%s?valor=%s&unidadInicial=%s&unidadFinal=%s",
                BASE_URL, tipo, valor,
                URLEncoder.encode(inicial, StandardCharsets.UTF_8),
                URLEncoder.encode(finalU, StandardCharsets.UTF_8));

        String respuesta = getJson(urlStr);
        String valorFinalStr = extraerCampoJson(respuesta, "valorFinal");
        if (valorFinalStr != null) {
```

```
            return Double.parseDouble(valorFinalStr);
        }
        throw new RuntimeException("No se encontrÃ³ 'valorFinal' en la respuesta del servidor: " + respuesta);
    } catch (Exception e) {
        throw new RuntimeException("Error en conversiÃ³n de " + tipo + ": " + e.getMessage(), e);
    }
  }

  private String getJson(String urlStr) throws Exception {
    URL url = new URL(urlStr);
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("GET");
    conn.setRequestProperty("Accept", "application/json");
    if (tokenSesion != null) {
        conn.setRequestProperty("Authorization", "Bearer " + tokenSesion);
    }

    int code = conn.getResponseCode();
    if (code != 200) {
        String errorMsg = leerStream(conn.getErrorStream());
        String cleanMsg = extraerCampoJson(errorMsg, "mensaje");
        if (cleanMsg != null && !cleanMsg.isBlank()) {
            throw new RuntimeException(cleanMsg);
        }
        throw new RuntimeException("HTTP " + code + ": " + errorMsg);
    }
    return leerStream(conn.getInputStream());
  }

  private String postJson(String urlStr, String jsonBody) throws Exception {
    URL url = new URL(urlStr);
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("POST");
    conn.setRequestProperty("Content-Type", "application/json; charset=UTF-8");
    conn.setRequestProperty("Accept", "application/json");
    if (tokenSesion != null) {
        conn.setRequestProperty("Authorization", "Bearer " + tokenSesion);
    }
    conn.setDoOutput(true);

    try (OutputStream os = conn.getOutputStream()) {
        byte[] input = jsonBody.getBytes(StandardCharsets.UTF_8);
        os.write(input, 0, input.length);
    }

    int code = conn.getResponseCode();
    if (code >= 400) {
        String errorMsg = leerStream(conn.getErrorStream());
```

```
            String cleanMsg = extraerCampoJson(errorMsg, "mensaje");
            if (cleanMsg != null && !cleanMsg.isBlank()) {
                throw new RuntimeException(cleanMsg);
            }
            throw new RuntimeException("HTTP " + code + ": " + errorMsg);
        }
        return leerStream(conn.getInputStream());
    }

    private String leerStream(java.io.InputStream is) throws Exception {
        if (is == null) return "";
        try (BufferedReader br = new BufferedReader(new InputStreamReader(is,
StandardCharsets.UTF_8))) {
            StringBuilder sb = new StringBuilder();
            String linea;
            while ((linea = br.readLine()) != null) {
                sb.append(linea.trim());
            }
            return sb.toString();
        }
    }

    private String extraerCampoJson(String json, String campo) {
        Pattern patternNumero = Pattern.compile("\"" + campo + "\"\\s*:\\s*([0-9.E+-]+)");
        Matcher matcherNumero = patternNumero.matcher(json);
        if (matcherNumero.find()) {
            return matcherNumero.group(1);
        }

        Pattern patternTexto = Pattern.compile("\"" + campo + "\"\\s*:\\s*\"([^\"]+)\"");
        Matcher matcherTexto = patternTexto.matcher(json);
        if (matcherTexto.find()) {
            return matcherTexto.group(1);
        }
        return null;
    }

    private String escapeJson(String texto) {
        if (texto == null) return "";
        return texto.replace("\\", "\\\\").replace("\"", "\\\"");
    }
}
```

4.2.5 Paso 5: Implementación de Controladores y Vista de Consola

ConsolaView presenta el menú interactivo, la lectura segura de contraseñas y el banner ASCII con el rostro oficial de James P. Sullivan:

Tabla 16

Vista de Consola con Banner ASCII (ConsolaView.java)

```
package ec.edu.monster.wsconversorcon.view;
```

```
import ec.edu.monster.wsconversorcon.model.UnidadLongitud;
import ec.edu.monster.wsconversorcon.model.UnidadMasa;
import ec.edu.monster.wsconversorcon.model.UnidadTemperatura;

import java.util.Scanner;

public class ConsolaView {

   private final Scanner scanner = new Scanner(System.in);

   public void mostrarBanner() {
      System.out.println("   /\\_______/\\");
      System.out.println("              /                 o                 o                  \\
â•”â••â••â••
```

```
public int leerEntero(String etiqueta, int min, int max) {
   while (true) {
      System.out.print(etiqueta + " [" + min + "-" + max + "]: ");
      String input = scanner.nextLine().trim();
      try {
         int valor = Integer.parseInt(input);
         if (valor >= min && valor <= max) {
            return valor;
         }
         System.out.println(" [!] La opciÃ³n debe estar entre " + min + " y " + max + ".");
      } catch (NumberFormatException e) {
         System.out.println(" [!] Ingrese un nÃºmero entero vÃ¡lido.");
      }
   }
}

public void mostrarMenuPrincipal() {
   System.out.println();
   System.out.println("--- MENÃš PRINCIPAL ---");
   System.out.println("1. Conversor de Longitudes");
   System.out.println("2. Conversor de Masa");
   System.out.println("3. Conversor de Temperatura");
   System.out.println("4. Salir");
}

public String seleccionarUnidadLongitud(String momento) {
   System.out.println("\nSeleccione Unidad " + momento + ":");
   UnidadLongitud[] valores = UnidadLongitud.values();
   for (int i = 0; i < valores.length; i++) {
      System.out.println((i + 1) + ". " + valores[i].name() + " (" + valores[i].getEtiqueta() + ")");
   }
   int seleccion = leerEntero("OpciÃ³n", 1, valores.length);
   return valores[seleccion - 1].name();
}

public String seleccionarUnidadMasa(String momento) {
   System.out.println("\nSeleccione Unidad " + momento + ":");
   UnidadMasa[] valores = UnidadMasa.values();
   for (int i = 0; i < valores.length; i++) {
      System.out.println((i + 1) + ". " + valores[i].name() + " (" + valores[i].getEtiqueta() + ")");
   }
   int seleccion = leerEntero("OpciÃ³n", 1, valores.length);
   return valores[seleccion - 1].name();
}

public String seleccionarUnidadTemperatura(String momento) {
   System.out.println("\nSeleccione Unidad " + momento + ":");
   UnidadTemperatura[] valores = UnidadTemperatura.values();
   for (int i = 0; i < valores.length; i++) {
      System.out.println((i + 1) + ". " + valores[i].name() + " (" + valores[i].getEtiqueta() + ")");
```

```
        }
        int seleccion = leerEntero("OpciÃ³n", 1, valores.length);
        return valores[seleccion - 1].name();
    }

    public void mostrarResultado(double resultado) {
        System.out.println();
        System.out.println("================================================");
        System.out.println("Resultado: " + resultado);
        System.out.println("================================================");
    }

    public void mostrarMensaje(String mensaje) {
        System.out.println(" > " + mensaje);
    }

    public void mostrarError(String error) {
        System.err.println(" [X] Error: " + error);
    }
}
```

ConsolaControlador orquesta el bucle principal de interacción y valida que el usuario inicie sesión antes de acceder al menú de cálculo:

Tabla 17

Controlador de Interfaz de Consola (ConsolaControlador.java)

```
package ec.edu.monster.wsconversorcon.controller;

import ec.edu.monster.wsconversorcon.view.ConsolaView;

public class ConsolaControlador {

    private final ConsolaView vista = new ConsolaView();
    private final AppControlador appControlador = new AppControlador();

    public void iniciar() {
        vista.mostrarBanner();

        if (!realizarLogin()) {
            vista.mostrarMensaje("Acceso cancelado. Finalizando programa.");
            return;
        }

        bucleMenuPrincipal();
    }

    private boolean realizarLogin() {
        System.out.println("\n--- INICIO DE SESIÃ“N ---");
        System.out.println("(Credenciales oficiales: MONSTER / MONSTER9)");

        while (true) {
```

```
        String usuario = vista.leerTexto("Usuario");
        String contrasenia = vista.leerContrasenia("ContraseÃ±a");

        if (usuario.isEmpty() || contrasenia.isEmpty()) {
            vista.mostrarMensaje("Por favor complete todos los campos.");
            continue;
        }

        try {
            String token = appControlador.login(usuario, contrasenia);
            if (token != null && !token.isBlank()) {
                vista.mostrarMensaje("Â¡Bienvenido " + usuario + "! SesiÃ³n iniciada
correctamente.");
                return true;
            } else {
                vista.mostrarError("Usuario o contraseÃ±a incorrectos.");
            }
        } catch (Exception e) {
            vista.mostrarError("No se pudo conectar con el servidor REST o credenciales
incorrectas: " + e.getMessage());
        }

        String reintentar = vista.leerTexto("Â¿Desea reintentar? (s/n)");
        if (!reintentar.equalsIgnoreCase("s")) {
            return false;
        }
    }
  }

  private void bucleMenuPrincipal() {
    boolean continuar = true;

    while (continuar) {
      vista.mostrarMenuPrincipal();
      int opcion = vista.leerEntero("Seleccione una opciÃ³n", 1, 4);

      switch (opcion) {
        case 1:
          procesarLongitud();
          break;
        case 2:
          procesarMasa();
          break;
        case 3:
          procesarTemperatura();
          break;
        case 4:
          continuar = false;
          vista.mostrarMensaje("Â¡Gracias por utilizar el WS Conversor! Hasta luego.");
          break;
```

```
        }
      }
    }

    private void procesarLongitud() {
      try {
        System.out.println("\n>>> CONVERSOR DE LONGITUDES <<<");
        String unidadInicial = vista.seleccionarUnidadLongitud("Inicial");
        double valor = vista.leerDouble("Ingrese el valor a convertir");
        String unidadFinal = vista.seleccionarUnidadLongitud("Final");

        double resultado = appControlador.convertirLongitud(valor, unidadInicial,
unidadFinal);
        vista.mostrarResultado(resultado);
      } catch (Exception e) {
        vista.mostrarError("Error en la conversiÃ³n: " + e.getMessage());
      }
    }

    private void procesarMasa() {
      try {
        System.out.println("\n>>> CONVERSOR DE MASA <<<");
        String unidadInicial = vista.seleccionarUnidadMasa("Inicial");
        double valor = vista.leerDouble("Ingrese el valor a convertir");
        String unidadFinal = vista.seleccionarUnidadMasa("Final");

        double resultado = appControlador.convertirMasa(valor, unidadInicial, unidadFinal);
        vista.mostrarResultado(resultado);
      } catch (Exception e) {
        vista.mostrarError("Error en la conversiÃ³n: " + e.getMessage());
      }
    }

    private void procesarTemperatura() {
      try {
        System.out.println("\n>>> CONVERSOR DE TEMPERATURA <<<");
        String unidadInicial = vista.seleccionarUnidadTemperatura("Inicial");
        double valor = vista.leerDouble("Ingrese el valor a convertir");
        String unidadFinal = vista.seleccionarUnidadTemperatura("Final");

        double resultado = appControlador.convertirTemperatura(valor, unidadInicial,
unidadFinal);
        vista.mostrarResultado(resultado);
      } catch (Exception e) {
        vista.mostrarError("Error en la conversiÃ³n: " + e.getMessage());
      }
    }
}
```

AppControlador actúa como mediador desacoplando la capa de vista del modelo:

Tabla 18

Controlador de Aplicación (AppControlador.java)

```
package ec.edu.monster.wsconversorcon.controller;

import ec.edu.monster.wsconversorcon.model.ConversorService;

public class AppControlador {

    private final ConversorService service = new ConversorService();

    public String login(String usuario, String contrasenia) {
        return service.login(usuario, contrasenia);
    }

    public double convertirLongitud(double valor, String unidadInicial, String unidadFinal) {
        return service.convertirLongitud(valor, unidadInicial, unidadFinal);
    }

    public double convertirMasa(double valor, String unidadInicial, String unidadFinal) {
        return service.convertirMasa(valor, unidadInicial, unidadFinal);
    }

    public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal)
{
        return service.convertirTemperatura(valor, unidadInicial, unidadFinal);
    }
}
```

Tabla 19

Clase Principal de Ejecución (WSConversorConsoleClient.java)

```
package ec.edu.monster.wsconversorcon;

import ec.edu.monster.wsconversorcon.controller.ConsolaControlador;

public class WSConversorConsoleClient {

    public static void main(String[] args) {
        ConsolaControlador controlador = new ConsolaControlador();
        controlador.iniciar();
    }
}
```

Figura 15
Ejecución del Cliente Consola (Inicio de sesión con banner Sullivan)

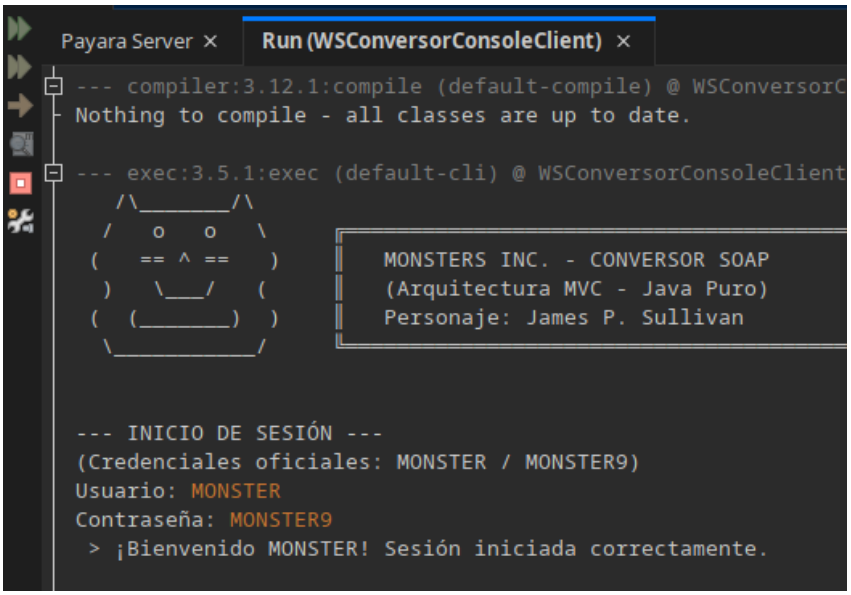
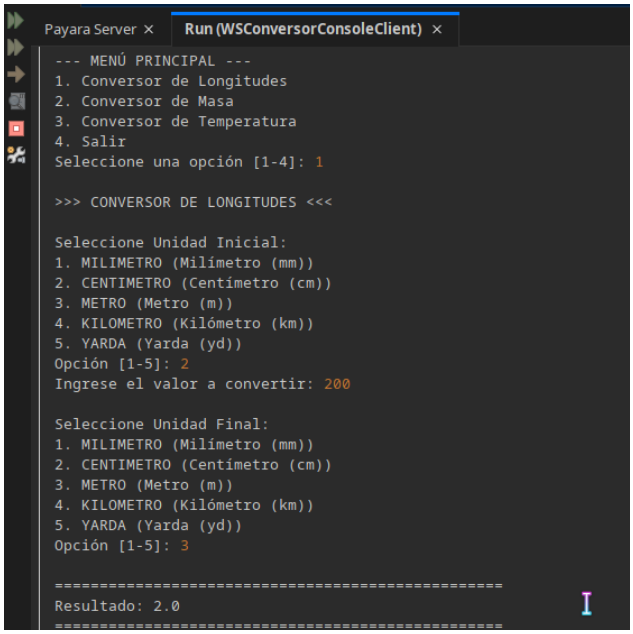


Figura 16
Ejecución de operaciones de conversión en Cliente Consola



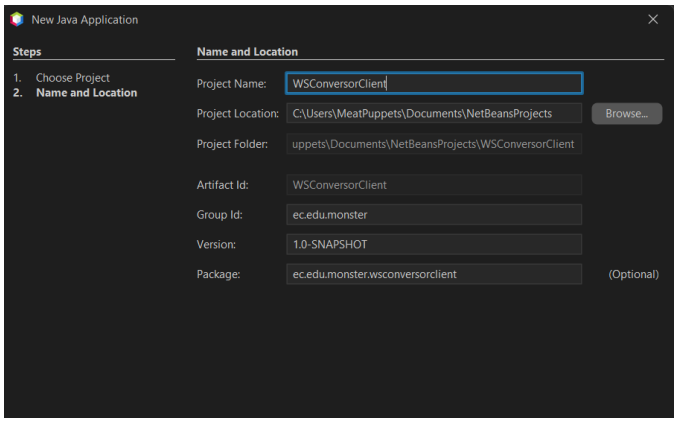
4.3 Cliente Escritorio

4.3.1 Paso 1: Creación del Proyecto Gráfico en NetBeans

Se crea un proyecto en NetBeans IDE denominado WSConversorClient para la integración visual de componentes Swing.

Figura 17

Creación del Proyecto Java Swing en NetBeans

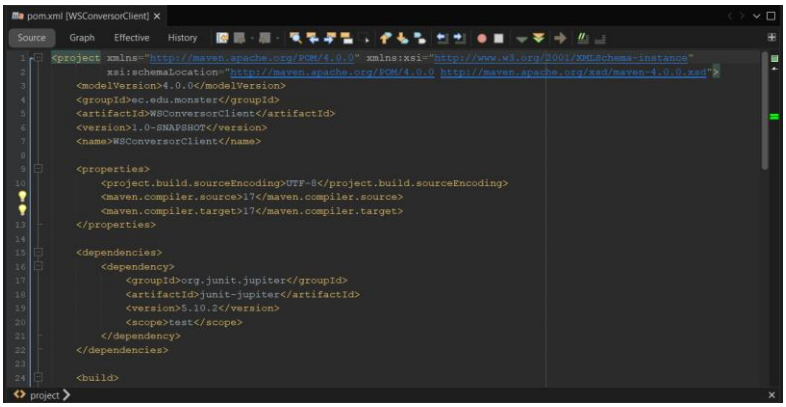


4.3.2 Paso 2: Configuración del Cliente HTTP y Modelos DTO

Se configuran los modelos de entidad y la clase de servicio HTTP sin necesidad de stubs SOAP generados por wsimport.

Figura 18

Configuración de módulos y dependencias en Cliente Escritorio

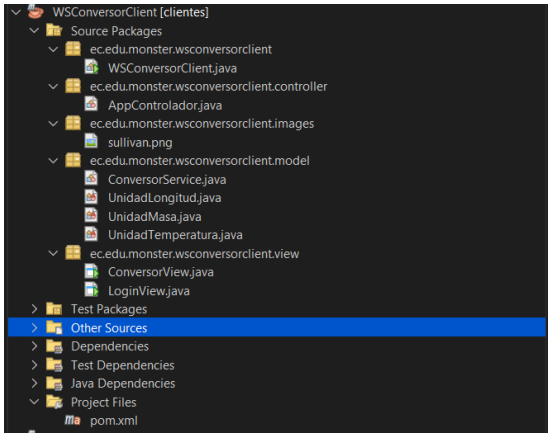


4.3.3 Paso 3: Estructuración de Paquetes MVC

Se distribuyen las clases en exactamente 3 paquetes: ec.edu.monster.wsconversorclient.model, ec.edu.monster.wsconversorclient.view y ec.edu.monster.wsconversorclient.controller.

Figura 19

Estructuración de paquetes bajo el patrón MVC en Cliente Escritorio



4.3.4 Paso 4: Implementación de la Capa de Modelo (ConversorService.java)

ConversorService gestiona el token de sesión en memoria y envía peticiones HTTP RESTful a /api/auth/login y /api/conversor/*:

Tabla 20

Capa de Modelo Cliente Escritorio (ConversorService.java)

```
package ec.edu.monster.wsconversorclient.model;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ConversorService {

   private static final String BASE_URL =
"http://localhost:8080/WS_ConversorUnidades_Java_REST/api";
   private static String tokenSesion;

   public static String getTokenSesion() {
      return tokenSesion;
   }

   public static void setTokenSesion(String token) {
      tokenSesion = token;
   }

   public String login(String usuario, String contrasenia) {
      try {
         String jsonInput = String.format("{\"usuario\":\"%s\",\"contrasenia\":\"%s\"}",
               escapeJson(usuario), escapeJson(contrasenia));

         String respuesta = postJson(BASE_URL + "/auth/login", jsonInput);
         String token = extraerCampoJson(respuesta, "token");
         if (token != null && !token.isBlank()) {
            tokenSesion = token;
            return token;
         }
         return null;
      } catch (Exception e) {
         throw new RuntimeException("Error al iniciar sesiÃ³n: " + e.getMessage(), e);
      }
   }
```

```
    public double convertirLongitud(double valor, String unidadInicial, String unidadFinal) {
        return ejecutarConversion("longitud", valor, unidadInicial, unidadFinal);
    }

    public double convertirMasa(double valor, String unidadInicial, String unidadFinal) {
        return ejecutarConversion("masa", valor, unidadInicial, unidadFinal);
    }

    public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal)
{
        return ejecutarConversion("temperatura", valor, unidadInicial, unidadFinal);
    }

    public String cambiarContrasenia(String actual, String nueva) {
        try {
            String                                          jsonInput                                          =
String.format("{\"contraseniaActual\":\"%s\",\"contraseniaNueva\":\"%s\"}",
                    escapeJson(actual), escapeJson(nueva));
            return postJson(BASE_URL + "/auth/cambiar-contrasenia", jsonInput);
        } catch (Exception e) {
            throw new RuntimeException("Error al cambiar contraseÃ±a: " + e.getMessage(), e);
        }
    }

    private double ejecutarConversion(String tipo, double valor, String inicial, String finalU) {
        try {
            String                                          urlStr                                          =
String.format("%s/conversor/%s?valor=%s&unidadInicial=%s&unidadFinal=%s",
                    BASE_URL, tipo, valor,
                    URLEncoder.encode(inicial, StandardCharsets.UTF_8),
                    URLEncoder.encode(finalU, StandardCharsets.UTF_8));

            String respuesta = getJson(urlStr);
            String valorFinalStr = extraerCampoJson(respuesta, "valorFinal");
            if (valorFinalStr != null) {
                return Double.parseDouble(valorFinalStr);
            }
            throw new RuntimeException("No se encontrÃ³ 'valorFinal' en la respuesta del
servidor: " + respuesta);
        } catch (Exception e) {
            throw new RuntimeException("Error en conversiÃ³n de " + tipo + ": " + e.getMessage(),
e);
        }
    }

    private String getJson(String urlStr) throws Exception {
        URL url = new URL(urlStr);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");
        conn.setRequestProperty("Accept", "application/json");
```

```
    if (tokenSesion != null) {
      conn.setRequestProperty("Authorization", "Bearer " + tokenSesion);
    }

    int code = conn.getResponseCode();
    if (code != 200) {
      String errorMsg = leerStream(conn.getErrorStream());
      String cleanMsg = extraerCampoJson(errorMsg, "mensaje");
      if (cleanMsg != null && !cleanMsg.isBlank()) {
        throw new RuntimeException(cleanMsg);
      }
      throw new RuntimeException("HTTP " + code + ": " + errorMsg);
    }
    return leerStream(conn.getInputStream());
  }

  private String postJson(String urlStr, String jsonBody) throws Exception {
    URL url = new URL(urlStr);
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("POST");
    conn.setRequestProperty("Content-Type", "application/json; charset=UTF-8");
    conn.setRequestProperty("Accept", "application/json");
    if (tokenSesion != null) {
      conn.setRequestProperty("Authorization", "Bearer " + tokenSesion);
    }
    conn.setDoOutput(true);

    try (OutputStream os = conn.getOutputStream()) {
      byte[] input = jsonBody.getBytes(StandardCharsets.UTF_8);
      os.write(input, 0, input.length);
    }

    int code = conn.getResponseCode();
    if (code >= 400) {
      String errorMsg = leerStream(conn.getErrorStream());
      String cleanMsg = extraerCampoJson(errorMsg, "mensaje");
      if (cleanMsg != null && !cleanMsg.isBlank()) {
        throw new RuntimeException(cleanMsg);
      }
      throw new RuntimeException("HTTP " + code + ": " + errorMsg);
    }
    return leerStream(conn.getInputStream());
  }

  private String leerStream(java.io.InputStream is) throws Exception {
    if (is == null) return "";
    try (BufferedReader br = new BufferedReader(new InputStreamReader(is,
StandardCharsets.UTF_8))) {
      StringBuilder sb = new StringBuilder();
      String linea;
```

```
        while ((linea = br.readLine()) != null) {
            sb.append(linea.trim());
        }
        return sb.toString();
    }
  }

  private String extraerCampoJson(String json, String campo) {
    Pattern patternNumero = Pattern.compile("\"" + campo + "\"\\s*:\\s*([0-9.E+-]+)");
    Matcher matcherNumero = patternNumero.matcher(json);
    if (matcherNumero.find()) {
      return matcherNumero.group(1);
    }

    Pattern patternTexto = Pattern.compile("\"" + campo + "\"\\s*:\\s*\"([^\"]+)\"");
    Matcher matcherTexto = patternTexto.matcher(json);
    if (matcherTexto.find()) {
      return matcherTexto.group(1);
    }
    return null;
  }

  private String escapeJson(String texto) {
    if (texto == null) return "";
    return texto.replace("\\", "\\\\").replace("\"", "\\\"");
  }
}
```

4.3.5 Paso 5: Diseño de Vistas con Formularios Swing

Se construyen LoginView y ConversorView utilizando componentes JFrame, visualizando la imagen corporativa de Sullivan, combos JComboBox dinámicos y paneles de resultado:

Figura 20

Diseño visual del formulario de login en Swing (LoginView)

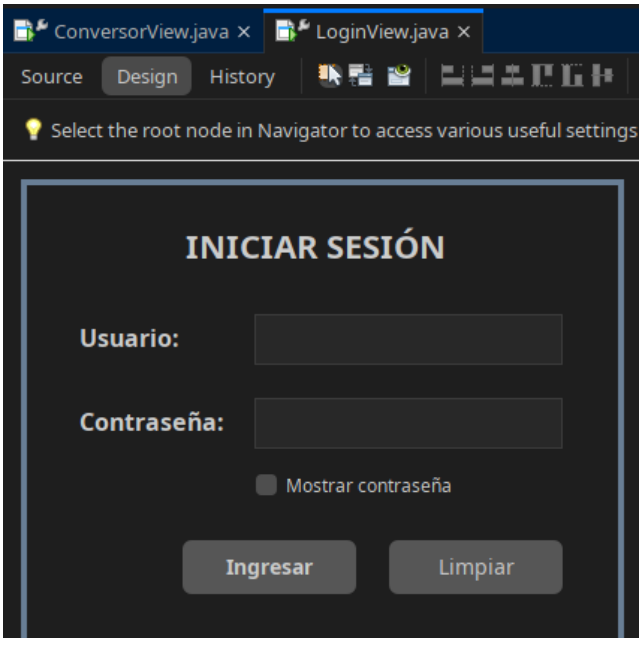
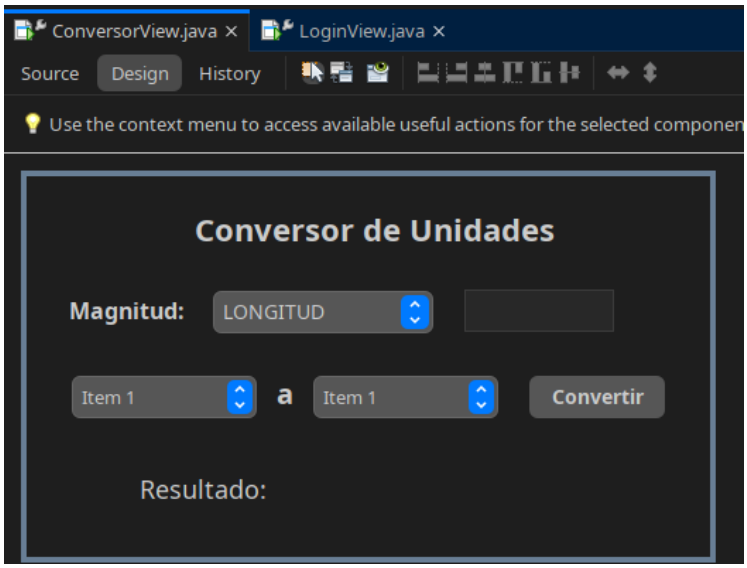


Figura 21
Diseño visual de la ventana principal del conversor (ConversorView)



4.3.6 Paso 6: Implementación del Controlador de Eventos (AppControlador.java)

AppControlador enlaza los eventos de clic en los botones de Swing con la ejecución síncrona en ConversorService:

Tabla 21

Controlador de Eventos de Escritorio (AppControlador.java)

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.monster.wsconversorclient.controller;

import ec.edu.monster.wsconversorclient.model.ConversorService;

/**
 *
 * @author meatpuppets
 */
public class AppControlador {

    private final ConversorService service = new ConversorService();

    public String login(String usuario, String contrasenia) {
        return service.login(usuario, contrasenia);
    }

    public double convertirLogintud(double n1, String unidadInicial, String unidadFinal){
        return service.convertirLongitud(n1, unidadInicial, unidadFinal);
    }
```

```
    public double convertirLongitud(double n1, String unidadInicial, String unidadFinal){
        return service.convertirLongitud(n1, unidadInicial, unidadFinal);
    }

    public double convertirMasa(double n1, String unidadInicial, String unidadFinal){
        return service.convertirMasa(n1, unidadInicial, unidadFinal);
    }

    public double convertirTemperatura(double n1, String unidadInicial, String unidadFinal){
        return service.convertirTemperatura(n1, unidadInicial, unidadFinal);
    }
}
```

Figura 22

Interfaz gráfica de Login ejecutada en Cliente Escritorio



Figura 23
Operación de conversión exitosa en Cliente Escritorio

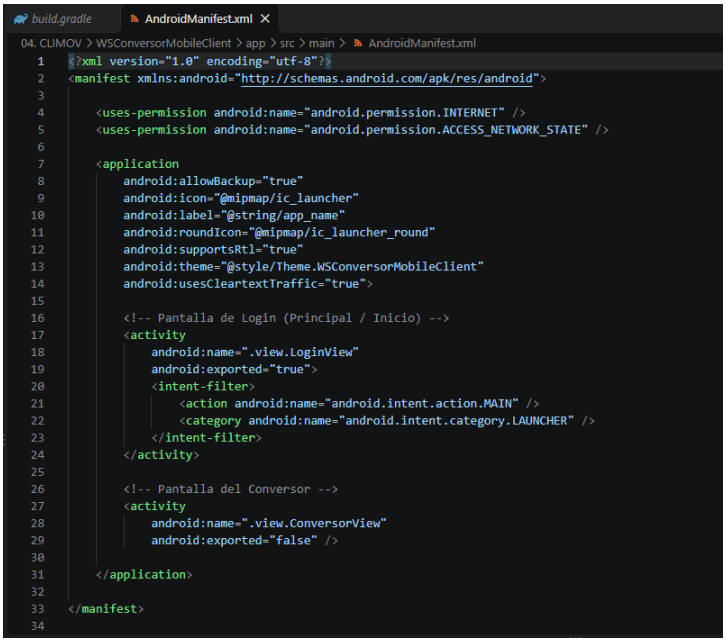


4.4 Cliente Móvil

4.4.1 Paso 1: Creación y Configuración del Proyecto Android en Gradle

Se configura el proyecto en Android Studio con Java nativo. En AndroidManifest.xml se habilitan los permisos de red (android.permission.INTERNET, ACCESS_NETWORK_STATE) y el soporte de tráfico local en texto claro (android:usesCleartextTraffic='true').

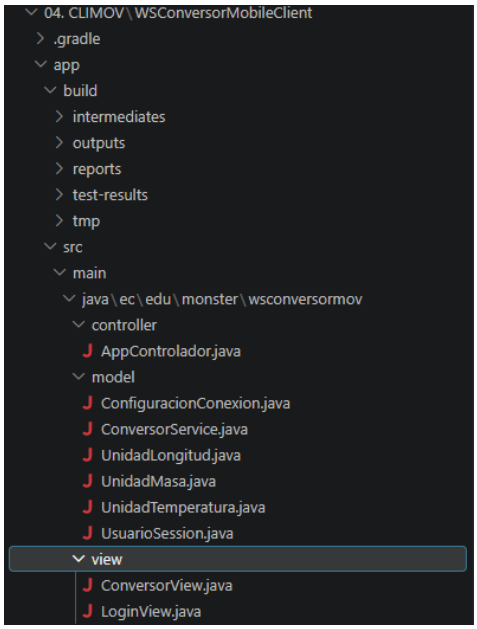
Figura 24
Configuración de permisos de red en AndroidManifest.xml



4.4.2 Paso 2: Estructuración Modular bajo el Patrón MVC

El código se estructura estrictamente en: ec.edu.monster.wsconversormov.model, ec.edu.monster.wsconversormov.view y ec.edu.monster.wsconversormov.controller.

Figura 25
Estructura modular de paquetes MVC en Cliente Móvil



4.4.3 Paso 3: Implementación de la Capa de Modelo y Despacho REST Asíncrono

Para evitar la excepción NetworkOnMainThreadException, ConversorService se ejecuta en segundo plano con ExecutorService, consumiendo los endpoints RESTful mediante HttpURLConnection y parseando la respuesta JSON:

Tabla 22

Modelo de comunicación REST asíncrona móvil (ConversorService.java)

```
package ec.edu.monster.wsconversormov.model;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ConversorService {

   private static String tokenSesion;

   public static String getTokenSesion() {
      return tokenSesion;
   }

   public static void setTokenSesion(String token) {
      tokenSesion = token;
```

```
    }

    public String login(String usuario, String contrasenia) throws Exception {
        String baseUrl = ConfiguracionConexion.getEndpointUrl();
        String jsonInput = String.format("{\"usuario\":\"%s\",\"contrasenia\":\"%s\"}",
                escapeJson(usuario), escapeJson(contrasenia));

        String respuesta = postJson(baseUrl + "/auth/login", jsonInput);
        String token = extraerCampoJson(respuesta, "token");
        if (token != null && !token.isBlank()) {
            tokenSesion = token;
            return token;
        }
        throw new RuntimeException("Credenciales incorrectas");
    }

    public double convertirLongitud(double valor, String unidadInicial, String unidadFinal)
throws Exception {
        return ejecutarConversion("longitud", valor, unidadInicial, unidadFinal);
    }

    public double convertirMasa(double valor, String unidadInicial, String unidadFinal) throws
Exception {
        return ejecutarConversion("masa", valor, unidadInicial, unidadFinal);
    }

    public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal)
throws Exception {
        return ejecutarConversion("temperatura", valor, unidadInicial, unidadFinal);
    }

    public String cambiarContrasenia(String actual, String nueva) throws Exception {
        String baseUrl = ConfiguracionConexion.getEndpointUrl();
        String                                    jsonInput                                    =
String.format("{\"contraseniaActual\":\"%s\",\"contraseniaNueva\":\"%s\"}",
                escapeJson(actual), escapeJson(nueva));
        return postJson(baseUrl + "/auth/cambiar-contrasenia", jsonInput);
    }

    private double ejecutarConversion(String tipo, double valor, String inicial, String finalU)
throws Exception {
        String baseUrl = ConfiguracionConexion.getEndpointUrl();
        String                                    urlStr                                    =
String.format("%s/conversor/%s?valor=%s&unidadInicial=%s&unidadFinal=%s",
                baseUrl, tipo, valor,
                URLEncoder.encode(inicial, "UTF-8"),
                URLEncoder.encode(finalU, "UTF-8"));

        String respuesta = getJson(urlStr);
        String valorFinalStr = extraerCampoJson(respuesta, "valorFinal");
```

```
    if (valorFinalStr != null) {
        return Double.parseDouble(valorFinalStr);
    }
    throw new RuntimeException("Respuesta inesperada del servidor: " + respuesta);
}

private String getJson(String urlStr) throws Exception {
    URL url = new URL(urlStr);
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("GET");
    conn.setRequestProperty("Accept", "application/json");
    conn.setConnectTimeout(8000);
    conn.setReadTimeout(8000);

    if (tokenSesion != null) {
        conn.setRequestProperty("Authorization", "Bearer " + tokenSesion);
    }

    int code = conn.getResponseCode();
    if (code != 200) {
        String errorMsg = leerStream(conn.getErrorStream());
        String cleanMsg = extraerCampoJson(errorMsg, "mensaje");
        if (cleanMsg != null && !cleanMsg.isBlank()) {
            throw new RuntimeException(cleanMsg);
        }
        throw new RuntimeException("HTTP " + code + ": " + errorMsg);
    }
    return leerStream(conn.getInputStream());
}

private String postJson(String urlStr, String jsonBody) throws Exception {
    URL url = new URL(urlStr);
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("POST");
    conn.setRequestProperty("Content-Type", "application/json; charset=UTF-8");
    conn.setRequestProperty("Accept", "application/json");
    conn.setConnectTimeout(8000);
    conn.setReadTimeout(8000);
    conn.setDoOutput(true);

    if (tokenSesion != null) {
        conn.setRequestProperty("Authorization", "Bearer " + tokenSesion);
    }

    try (OutputStream os = conn.getOutputStream()) {
        byte[] input = jsonBody.getBytes(StandardCharsets.UTF_8);
        os.write(input, 0, input.length);
    }

    int code = conn.getResponseCode();
```

```
        if (code >= 400) {
            String errorMsg = leerStream(conn.getErrorStream());
            String cleanMsg = extraerCampoJson(errorMsg, "mensaje");
            if (cleanMsg != null && !cleanMsg.isBlank()) {
                throw new RuntimeException(cleanMsg);
            }
            throw new RuntimeException("HTTP " + code + ": " + errorMsg);
        }
        return leerStream(conn.getInputStream());
    }

    private String leerStream(java.io.InputStream is) throws Exception {
        if (is == null) return "";
        try (BufferedReader br = new BufferedReader(new InputStreamReader(is,
StandardCharsets.UTF_8))) {
            StringBuilder sb = new StringBuilder();
            String linea;
            while ((linea = br.readLine()) != null) {
                sb.append(linea.trim());
            }
            return sb.toString();
        }
    }

    private String extraerCampoJson(String json, String campo) {
        Pattern patternNumero = Pattern.compile("\"" + campo + "\"\\s*:\\s*([0-9.E+-]+)");
        Matcher matcherNumero = patternNumero.matcher(json);
        if (matcherNumero.find()) {
            return matcherNumero.group(1);
        }

        Pattern patternTexto = Pattern.compile("\"" + campo + "\"\\s*:\\s*\"([^\"]+)\"");
        Matcher matcherTexto = patternTexto.matcher(json);
        if (matcherTexto.find()) {
            return matcherTexto.group(1);
        }
        return null;
    }

    private String escapeJson(String texto) {
        if (texto == null) return "";
        return texto.replace("\\", "\\\\").replace("\"", "\\\"");
    }
}
```

4.4.4 Paso 4: Diseño de Layouts XML

Se diseñan las interfaces gráficas en XML con diseño temático de Monsters Inc:

Figura 26

Diseño visual de layout de login móvil (activity_login.xml)

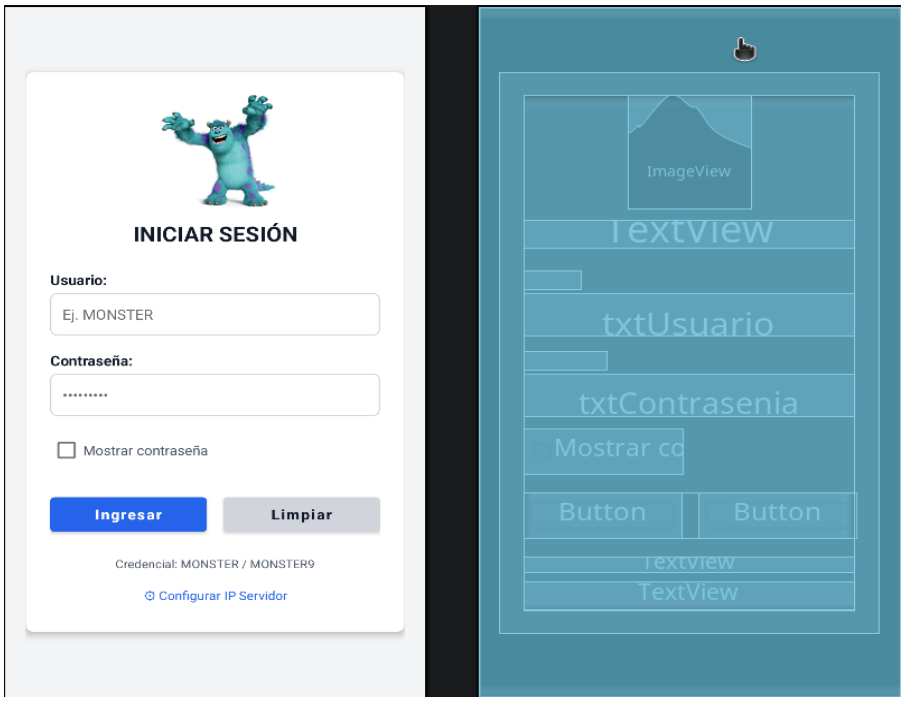


Figura 27

Diseño visual de layout del conversor móvil (activity_conversor.xml)

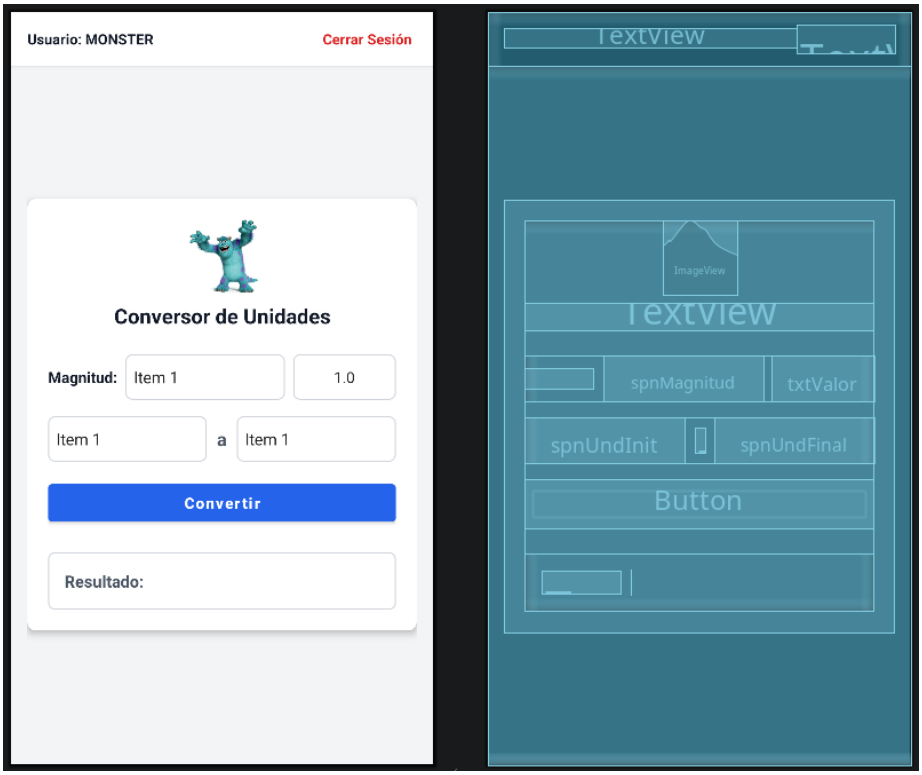


Tabla 23

Diseño XML de la vista de autenticación móvil

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/background"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:orientation="vertical"
        android:padding="20dp">

        <!-- Card estilo ventana sencilla -->
        <androidx.cardview.widget.CardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            app:cardBackgroundColor="@color/card_bg"
            app:cardCornerRadius="8dp"
            app:cardElevation="4dp"
            app:contentPadding="24dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical">

                <!-- Logo Sulley Monsters Inc -->
                <ImageView
                    android:id="@+id/imgSulley"
                    android:layout_width="120dp"
                    android:layout_height="120dp"
                    android:layout_gravity="center_horizontal"
                    android:layout_marginBottom="12dp"
                    android:contentDescription="@string/app_name"
                    android:src="@drawable/sullivan" />

                <TextView
                    android:id="@+id/lblTituloLogin"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:layout_marginBottom="24dp"
                    android:gravity="center"
                    android:text="@string/title_login"
                    android:textColor="@color/text_primary"
                    android:textSize="22sp"
                    android:textStyle="bold" />

                <!-- Campo Usuario -->
```

```
<TextView
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:layout_marginBottom="4dp"
   android:text="@string/prompt_usuario"
   android:textColor="@color/text_primary"
   android:textSize="15sp"
   android:textStyle="bold" />

<EditText
   android:id="@+id/txtUsuario"
   android:layout_width="match_parent"
   android:layout_height="45dp"
   android:layout_marginBottom="16dp"
   android:background="@drawable/edittext_border"
   android:hint="Ej. MONSTER"
   android:inputType="text"
   android:paddingHorizontal="12dp"
   android:textSize="15sp" />

<!-- Campo ContraseÃ±a -->
<TextView
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:layout_marginBottom="4dp"
   android:text="@string/prompt_contrasenia"
   android:textColor="@color/text_primary"
   android:textSize="15sp"
   android:textStyle="bold" />

<EditText
   android:id="@+id/txtContrasenia"
   android:layout_width="match_parent"
   android:layout_height="45dp"
   android:layout_marginBottom="12dp"
   android:background="@drawable/edittext_border"
   android:hint="â€¢â€¢â€¢â€¢â€¢â€¢â€¢â€¢"
   android:inputType="textPassword"
   android:paddingHorizontal="12dp"
   android:textSize="15sp" />

<!-- Checkbox Mostrar contraseÃ±a -->
<CheckBox
   android:id="@+id/chkMostrarContrasenia"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:layout_marginBottom="20dp"
   android:text="@string/chk_mostrar_contrasenia"
   android:textColor="@color/text_secondary"
   android:textSize="14sp" />
```

```
<!-- Botones Ingresar y Limpiar -->
<LinearLayout
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:gravity="center"
   android:orientation="horizontal">

   <Button
      android:id="@+id/btnIngresar"
      android:layout_width="0dp"
      android:layout_height="48dp"
      android:layout_marginEnd="8dp"
      android:layout_weight="1"
      android:backgroundTint="@color/primary"
      android:text="@string/btn_ingresar"
      android:textAllCaps="false"
      android:textColor="@color/card_bg"
      android:textSize="15sp"
      android:textStyle="bold" />

   <Button
      android:id="@+id/btnLimpiar"
      android:layout_width="0dp"
      android:layout_height="48dp"
      android:layout_marginStart="8dp"
      android:layout_weight="1"
      android:backgroundTint="@color/border"
      android:text="@string/btn_limpiar"
      android:textAllCaps="false"
      android:textColor="@color/text_primary"
      android:textSize="15sp"
      android:textStyle="bold" />
</LinearLayout>

<TextView
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:layout_marginTop="20dp"
   android:gravity="center"
   android:text="Credencial: MONSTER / MONSTER9"
   android:textColor="@color/text_secondary"
   android:textSize="12sp" />

<!-- BotÃ³n para configurar IP del servidor -->
<TextView
   android:id="@+id/btnConfigIp"
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:layout_marginTop="10dp"
```

```
            android:gravity="center"
            android:padding="6dp"
            android:text="@string/btn_config_ip"
            android:textColor="@color/primary"
            android:textSize="13sp" />

        </LinearLayout>
    </androidx.cardview.widget.CardView>

  </LinearLayout>
</ScrollView>
```

Tabla 24

Diseño XML de la vista principal móvil

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:background="@color/background"
  android:orientation="vertical">

  <!-- Barra superior simple: Usuario y Cerrar SesiÃ³n -->
  <LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="@color/card_bg"
    android:elevation="2dp"
    android:gravity="center_vertical"
    android:orientation="horizontal"
    android:paddingHorizontal="16dp"
    android:paddingVertical="12dp">

    <TextView
      android:id="@+id/txtUsuarioSesion"
      android:layout_width="0dp"
      android:layout_height="wrap_content"
      android:layout_weight="1"
      android:text="Usuario: MONSTER"
      android:textColor="@color/text_primary"
      android:textSize="14sp"
      android:textStyle="bold" />

    <TextView
      android:id="@+id/btnCerrarSesion"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:padding="4dp"
      android:text="@string/btn_cerrar_sesion"
      android:textColor="@color/danger"
```

```
        android:textSize="14sp"
        android:textStyle="bold" />
</LinearLayout>

<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:orientation="vertical"
        android:padding="16dp">

        <!-- Card tipo ventana de escritorio -->
        <androidx.cardview.widget.CardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            app:cardBackgroundColor="@color/card_bg"
            app:cardCornerRadius="8dp"
            app:cardElevation="4dp"
            app:contentPadding="20dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical">

                <!-- Logo Sulley Monsters Inc -->
                <ImageView
                    android:id="@+id/imgSulleyConversor"
                    android:layout_width="72dp"
                    android:layout_height="72dp"
                    android:layout_gravity="center_horizontal"
                    android:layout_marginBottom="8dp"
                    android:contentDescription="@string/app_name"
                    android:src="@drawable/sullivan" />

                <!-- TÃ-tulo dinÃ¡mico -->
                <TextView
                    android:id="@+id/txtTitulo"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:layout_marginBottom="24dp"
                    android:gravity="center"
                    android:text="@string/title_conversor"
                    android:textColor="@color/text_primary"
                    android:textSize="20sp"
```

```
    android:textStyle="bold" />

  <!-- Fila 1: Magnitud y Valor -->
  <LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    android:gravity="center_vertical"
    android:orientation="horizontal">

    <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_marginEnd="8dp"
      android:text="@string/prompt_magnitud"
      android:textColor="@color/text_primary"
      android:textSize="15sp"
      android:textStyle="bold" />

    <Spinner
      android:id="@+id/spnMagnitud"
      android:layout_width="0dp"
      android:layout_height="44dp"
      android:layout_marginEnd="8dp"
      android:layout_weight="1"
      android:background="@drawable/edittext_border" />

    <EditText
      android:id="@+id/txtValor"
      android:layout_width="100dp"
      android:layout_height="44dp"
      android:background="@drawable/edittext_border"
      android:gravity="center"
      android:hint="1.0"
      android:inputType="numberDecimal"
      android:paddingHorizontal="8dp"
      android:text="1.0"
      android:textSize="15sp" />
  </LinearLayout>

  <!-- Fila 2: undInit  a  undFinal -->
  <LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    android:gravity="center_vertical"
    android:orientation="horizontal">

    <Spinner
      android:id="@+id/spnUndInit"
```

```
            android:layout_width="0dp"
            android:layout_height="44dp"
            android:layout_weight="1"
            android:background="@drawable/edittext_border" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginHorizontal="10dp"
            android:text="@string/prompt_a"
            android:textColor="@color/text_secondary"
            android:textSize="18sp"
            android:textStyle="bold" />

        <Spinner
            android:id="@+id/spnUndFinal"
            android:layout_width="0dp"
            android:layout_height="44dp"
            android:layout_weight="1"
            android:background="@drawable/edittext_border" />
    </LinearLayout>

    <!-- BotÃ³n Convertir -->
    <Button
        android:id="@+id/btnConvertir"
        android:layout_width="match_parent"
        android:layout_height="48dp"
        android:layout_marginBottom="24dp"
        android:backgroundTint="@color/primary"
        android:text="@string/btn_convertir"
        android:textAllCaps="false"
        android:textColor="@color/card_bg"
        android:textSize="15sp"
        android:textStyle="bold" />

    <!-- Fila 3: Resultado: [ conversionResultado ] -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@drawable/edittext_border"
        android:gravity="center_vertical"
        android:orientation="horizontal"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginEnd="10dp"
            android:text="@string/label_resultado"
            android:textColor="@color/text_secondary"
```

```
                android:textSize="16sp"
                android:textStyle="bold" />

            <TextView
                android:id="@+id/txtResultado"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text=""
                android:textColor="@color/accent"
                android:textSize="18sp"
                android:textStyle="bold" />
        </LinearLayout>

    </LinearLayout>
  </androidx.cardview.widget.CardView>

 </LinearLayout>
</ScrollView>

</LinearLayout>
```

4.4.5 Paso 5: Implementación de Actividades y Controlador Móvil

LoginView y ConversorView coordinan la interfaz mediante AppControlador:

Tabla 25

Controlador de actividades móviles (AppControlador.java)

```
package ec.edu.monster.wsconversormov.controller;

import ec.edu.monster.wsconversormov.model.ConversorService;

public class AppControlador {

   private final ConversorService service = new ConversorService();

   public String login(String usuario, String contrasenia) throws Exception {
      return service.login(usuario, contrasenia);
   }

   public double convertirLongitud(double valor, String unidadInicial, String unidadFinal)
throws Exception {
      return service.convertirLongitud(valor, unidadInicial, unidadFinal);
   }

   public double convertirMasa(double valor, String unidadInicial, String unidadFinal) throws
Exception {
      return service.convertirMasa(valor, unidadInicial, unidadFinal);
   }

   public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal)
throws Exception {
      return service.convertirTemperatura(valor, unidadInicial, unidadFinal);
```

```
    }
}
```

Figura 28

Ejecución del Cliente Móvil (Login con imagen Sullivan)

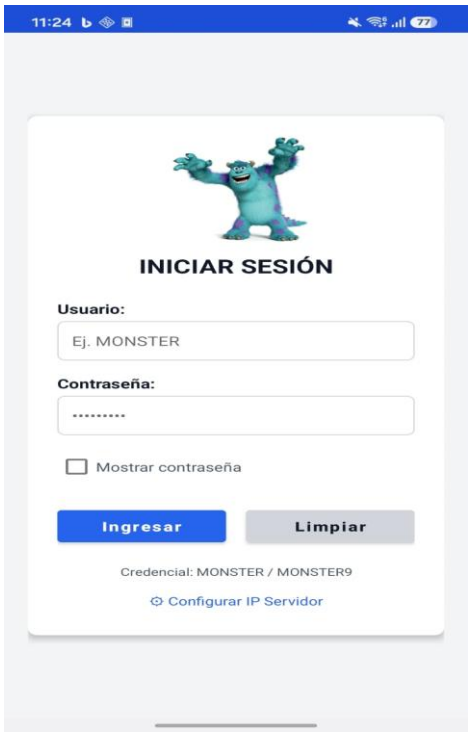
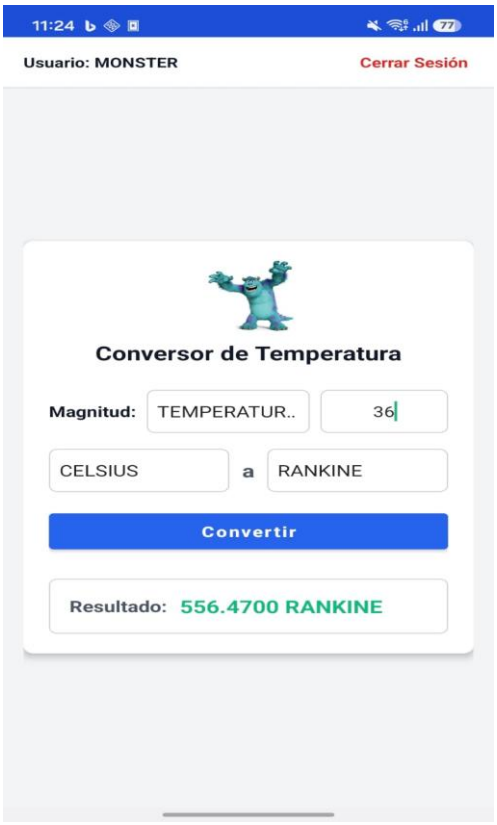


Figura 29
Ejecución del Cliente Móvil (Conversión y resultados dinámicos)

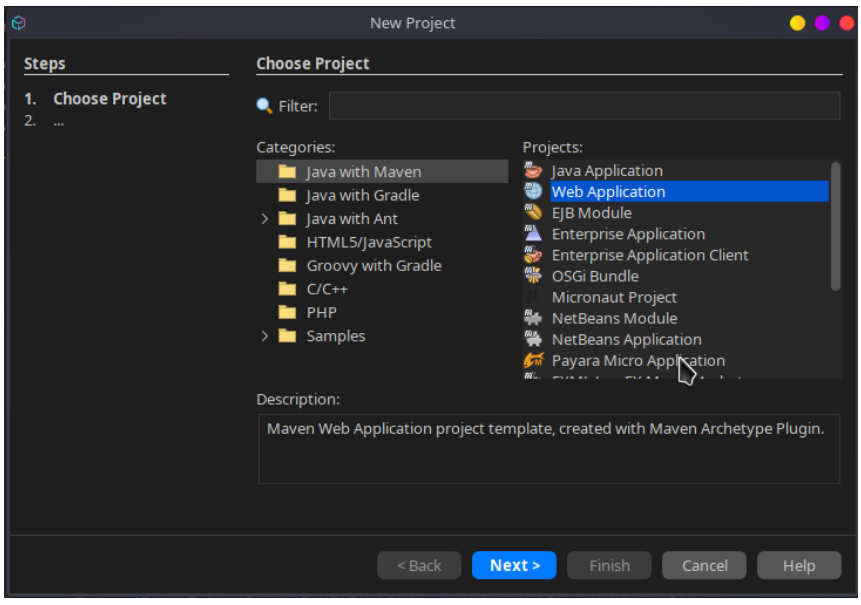


4.5 Cliente Web

4.5.1 Paso 1: Creación del Proyecto Web Jakarta Faces en NetBeans

Se crea el proyecto Maven WSConversorWebClient configurando el soporte de Jakarta Faces 4.0 (JSF) sobre Payara Server.

Figura 30
Creación del proyecto Web Jakarta Faces en NetBeans



4.5.2 Paso 2: Configuración del Entorno JSF y Descriptores

En WEB-INF se declaran web.xml (mapeo de FacesServlet con *.xhtml), beans.xml (activación de CDI) y faces-config.xml:

Figura 31

Descriptores de despliegue web.xml, beans.xml y faces-config.xml

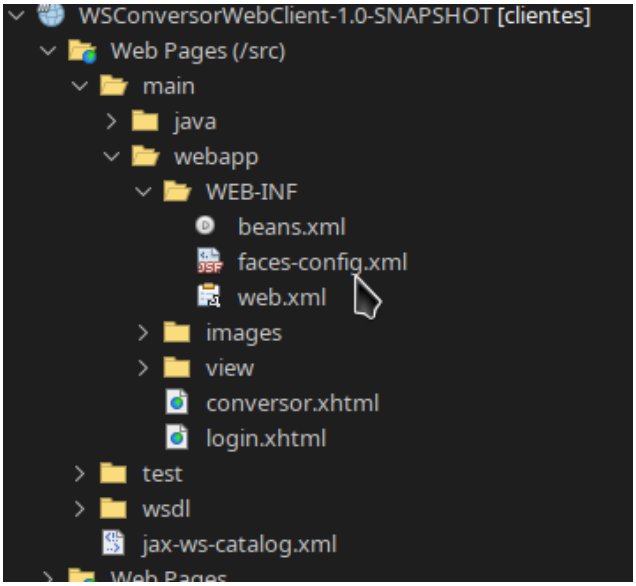


Tabla 26

Descriptores de despliegue web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
        version="6.0">

    <display-name>WSConversorWebClient</display-name>

    <!-- Jakarta Faces Servlet -->
    <servlet>
        <servlet-name>Faces Servlet</servlet-name>
        <servlet-class>jakarta.faces.webapp.FacesServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>Faces Servlet</servlet-name>
        <url-pattern>*.xhtml</url-pattern>
    </servlet-mapping>

    <welcome-file-list>
        <welcome-file>login.xhtml</welcome-file>
    </welcome-file-list>

    <session-config>
```

```
    <session-timeout>30</session-timeout>
  </session-config>

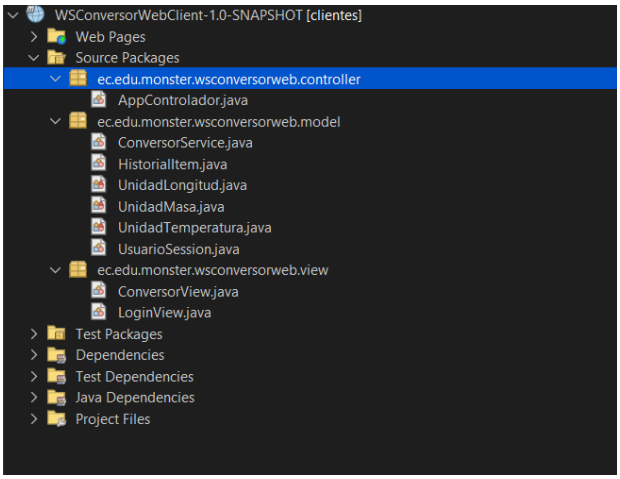
</web-app>
```

4.5.3 Paso 3: Estructuración MVC en Jakarta Faces

Se organizan los paquetes en ec.edu.monster.wsconversorweb.model, ec.edu.monster.wsconversorweb.view y ec.edu.monster.wsconversorweb.controller, sin necesidad de stubs SOAP.

Figura 32

Estructuración de paquetes MVC en Cliente Web



4.5.4 Paso 4: Implementación de la Capa de Modelo (ConversorService.java)

ConversorService gestiona el consumo de la API RESTful de Payara Server mediante HttpURLConnection y serialización JSON:

Tabla 27

Modelo de consumo REST en Cliente Web (ConversorService.java)

```
package ec.edu.monster.wsconversorweb.model;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.io.Serializable;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ConversorService implements Serializable {

   private static final String BASE_URL =
"http://localhost:8080/WS_ConversorUnidades_Java_REST/api";
   private static String tokenSesion;
```

```
    public static String getTokenSesion() {
        return tokenSesion;
    }

    public static void setTokenSesion(String token) {
        tokenSesion = token;
    }

    public String login(String usuario, String contrasenia) {
        try {
            String jsonInput = String.format("{\"usuario\":\"%s\",\"contrasenia\":\"%s\"}",
                    escapeJson(usuario), escapeJson(contrasenia));

            String respuesta = postJson(BASE_URL + "/auth/login", jsonInput);
            String token = extraerCampoJson(respuesta, "token");
            if (token != null && !token.isBlank()) {
                tokenSesion = token;
                return token;
            }
            return null;
        } catch (Exception e) {
            throw new RuntimeException("Error al iniciar sesiÃ³n: " + e.getMessage(), e);
        }
    }

    public double convertirLongitud(double valor, String unidadInicial, String unidadFinal) {
        return ejecutarConversion("longitud", valor, unidadInicial, unidadFinal);
    }

    public double convertirMasa(double valor, String unidadInicial, String unidadFinal) {
        return ejecutarConversion("masa", valor, unidadInicial, unidadFinal);
    }

    public double convertirTemperatura(double valor, String unidadInicial, String unidadFinal)
{
        return ejecutarConversion("temperatura", valor, unidadInicial, unidadFinal);
    }

    public String cambiarContrasenia(String actual, String nueva) {
        try {
            String                                          jsonInput                                          =
String.format("{\"contraseniaActual\":\"%s\",\"contraseniaNueva\":\"%s\"}",
                    escapeJson(actual), escapeJson(nueva));
            return postJson(BASE_URL + "/auth/cambiar-contrasenia", jsonInput);
        } catch (Exception e) {
            throw new RuntimeException("Error al cambiar contraseÃ±a: " + e.getMessage(), e);
        }
    }

    private double ejecutarConversion(String tipo, double valor, String inicial, String finalU) {
```

```
        try {
            String urlStr = String.format("%s/conversor/%s?valor=%s&unidadInicial=%s&unidadFinal=%s",
                    BASE_URL, tipo, valor,
                    URLEncoder.encode(inicial, StandardCharsets.UTF_8),
                    URLEncoder.encode(finalU, StandardCharsets.UTF_8));

            String respuesta = getJson(urlStr);
            String valorFinalStr = extraerCampoJson(respuesta, "valorFinal");
            if (valorFinalStr != null) {
                return Double.parseDouble(valorFinalStr);
            }
            throw new RuntimeException("No se encontrÃ³ 'valorFinal' en la respuesta del servidor: " + respuesta);
        } catch (Exception e) {
            throw new RuntimeException("Error en conversiÃ³n de " + tipo + ": " + e.getMessage(), e);
        }
    }

    private String getJson(String urlStr) throws Exception {
        URL url = new URL(urlStr);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");
        conn.setRequestProperty("Accept", "application/json");
        if (tokenSesion != null) {
            conn.setRequestProperty("Authorization", "Bearer " + tokenSesion);
        }

        int code = conn.getResponseCode();
        if (code != 200) {
            String errorMsg = leerStream(conn.getErrorStream());
            String cleanMsg = extraerCampoJson(errorMsg, "mensaje");
            if (cleanMsg != null && !cleanMsg.isBlank()) {
                throw new RuntimeException(cleanMsg);
            }
            throw new RuntimeException("HTTP " + code + ": " + errorMsg);
        }
        return leerStream(conn.getInputStream());
    }

    private String postJson(String urlStr, String jsonBody) throws Exception {
        URL url = new URL(urlStr);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json; charset=UTF-8");
        conn.setRequestProperty("Accept", "application/json");
        if (tokenSesion != null) {
            conn.setRequestProperty("Authorization", "Bearer " + tokenSesion);
        }
```

```
        conn.setDoOutput(true);

        try (OutputStream os = conn.getOutputStream()) {
            byte[] input = jsonBody.getBytes(StandardCharsets.UTF_8);
            os.write(input, 0, input.length);
        }

        int code = conn.getResponseCode();
        if (code >= 400) {
            String errorMsg = leerStream(conn.getErrorStream());
            String cleanMsg = extraerCampoJson(errorMsg, "mensaje");
            if (cleanMsg != null && !cleanMsg.isBlank()) {
                throw new RuntimeException(cleanMsg);
            }
            throw new RuntimeException("HTTP " + code + ": " + errorMsg);
        }
        return leerStream(conn.getInputStream());
    }

    private String leerStream(java.io.InputStream is) throws Exception {
        if (is == null) return "";
        try (BufferedReader br = new BufferedReader(new InputStreamReader(is,
StandardCharsets.UTF_8))) {
            StringBuilder sb = new StringBuilder();
            String linea;
            while ((linea = br.readLine()) != null) {
                sb.append(linea.trim());
            }
            return sb.toString();
        }
    }

    private String extraerCampoJson(String json, String campo) {
        Pattern patternNumero = Pattern.compile("\"" + campo + "\"\\s*:\\s*([0-9.E+-]+)");
        Matcher matcherNumero = patternNumero.matcher(json);
        if (matcherNumero.find()) {
            return matcherNumero.group(1);
        }

        Pattern patternTexto = Pattern.compile("\"" + campo + "\"\\s*:\\s*\"([^\"]+)\"");
        Matcher matcherTexto = patternTexto.matcher(json);
        if (matcherTexto.find()) {
            return matcherTexto.group(1);
        }
        return null;
    }

    private String escapeJson(String texto) {
        if (texto == null) return "";
        return texto.replace("\\", "\\\\").replace("\"", "\\\"");
```

```
    }
}
```

4.5.5 Paso 5: Implementación de Managed Beans Controladores con CDI

ConversorView implementa la lógica de presentación con @Named y @SessionScoped, coordinando la reactividad Ajax sin JavaScript externo:

Tabla 28

Managed Bean reactivo con CDI (ConversorView.java)

```
package ec.edu.monster.wsconversorweb.view;

import ec.edu.monster.wsconversorweb.controller.AppControlador;
import ec.edu.monster.wsconversorweb.model.HistorialItem;
import ec.edu.monster.wsconversorweb.model.UnidadLongitud;
import ec.edu.monster.wsconversorweb.model.UnidadMasa;
import ec.edu.monster.wsconversorweb.model.UnidadTemperatura;
import jakarta.annotation.PostConstruct;
import jakarta.enterprise.context.SessionScoped;
import jakarta.faces.application.FacesMessage;
import jakarta.faces.context.FacesContext;
import jakarta.inject.Inject;
import jakarta.inject.Named;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * Vista / Bean del Conversor de Unidades para Cliente Web (PatrÃ³n MVC).
 */
@Named("conversorView")
@SessionScoped
public class ConversorView implements Serializable {

    private static final long serialVersionUID = 1L;

    @Inject
    private LoginView loginView;

    private String tipoMagnitud = "LONGITUD";
    private Double valor;
    private String unidadOrigen;
    private String unidadDestino;
    private Double resultado;
    private final List<HistorialItem> historial = new ArrayList<>();
    private final AppControlador controlador = new AppControlador();

    @PostConstruct
    public void init() {
        actualizarUnidades();
    }
```

```
public void verificarSesion() {
    if (loginView == null || !loginView.isAutenticado()) {
        try {
            FacesContext.getCurrentInstance().getExternalContext().redirect("login.xhtml");
        } catch (Exception ignored) {
        }
    }
}

public void cambiarMagnitud() {
    actualizarUnidades();
    resultado = null;
}

private void actualizarUnidades() {
    List<String> disponibles = getUnidadesDisponibles();
    if (!disponibles.isEmpty()) {
        unidadOrigen = disponibles.get(0);
        unidadDestino = disponibles.size() > 1 ? disponibles.get(1) : disponibles.get(0);
    }
}

public List<String> getUnidadesDisponibles() {
    switch (tipoMagnitud) {
        case "MASA":
            return Arrays.stream(UnidadMasa.values()).map(Enum::name).toList();
        case "TEMPERATURA":
            return Arrays.stream(UnidadTemperatura.values()).map(Enum::name).toList();
        case "LONGITUD":
        default:
            return Arrays.stream(UnidadLongitud.values()).map(Enum::name).toList();
    }
}

public String getTitulo() {
    switch (tipoMagnitud) {
        case "MASA":
            return "Conversor de Masa";
        case "TEMPERATURA":
            return "Conversor de Temperatura";
        case "LONGITUD":
        default:
            return "Conversor de Longitudes";
    }
}

public void convertir() {
    if (loginView == null || !loginView.isAutenticado()) {
        FacesContext.getCurrentInstance().addMessage(null,
```

```
            new FacesMessage(FacesMessage.SEVERITY_ERROR, "Debe iniciar sesiÃ³n primero.", null));
            return;
        }

        if (valor == null) {
            FacesContext.getCurrentInstance().addMessage(null,
                new FacesMessage(FacesMessage.SEVERITY_ERROR, "Por favor ingrese un valor numÃ©rico.", null));
            return;
        }

        try {
            double res;
            switch (tipoMagnitud) {
                case "MASA":
                    res = controlador.convertirMasa(valor, unidadOrigen, unidadDestino);
                    break;
                case "TEMPERATURA":
                    res = controlador.convertirTemperatura(valor, unidadOrigen, unidadDestino);
                    break;
                case "LONGITUD":
                default:
                    res = controlador.convertirLongitud(valor, unidadOrigen, unidadDestino);
                    break;
            }
            resultado = res;
            historial.add(0, new HistorialItem(tipoMagnitud, valor, unidadOrigen, res, unidadDestino));
        } catch (Exception e) {
            FacesContext.getCurrentInstance().addMessage(null,
                new FacesMessage(FacesMessage.SEVERITY_ERROR, "Error en conversiÃ³n: " + e.getMessage(), null));
        }
    }

    public void limpiar() {
        valor = null;
        resultado = null;
        actualizarUnidades();
    }

    public String logout() {
        return loginView.logout();
    }

    public String getTipoMagnitud() {
        return tipoMagnitud;
    }
```

```
    public void setTipoMagnitud(String tipoMagnitud) {
        this.tipoMagnitud = tipoMagnitud;
    }

    public Double getValor() {
        return valor;
    }

    public void setValor(Double valor) {
        this.valor = valor;
    }

    public String getUnidadOrigen() {
        return unidadOrigen;
    }

    public void setUnidadOrigen(String unidadOrigen) {
        this.unidadOrigen = unidadOrigen;
    }

    public String getUnidadDestino() {
        return unidadDestino;
    }

    public void setUnidadDestino(String unidadDestino) {
        this.unidadDestino = unidadDestino;
    }

    public Double getResultado() {
        return resultado;
    }

    public void setResultado(Double resultado) {
        this.resultado = resultado;
    }

    public List<HistorialItem> getHistorial() {
        return historial;
    }

    public LoginView getLoginView() {
        return loginView;
    }

    public void setLoginView(LoginView loginView) {
        this.loginView = loginView;
    }
}
```

4.5.6 Paso 6: Diseño de Vistas Web con Facelets (XHTML)

login.xhtml y conversor.xhtml implementan interfaces puras Facelets con componentes estándar <h:form>, <h:selectOneMenu> y llamadas asíncronas <f:ajax>:

Tabla 29

Formulario Facelets de autenticación web (login.xhtml)

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="jakarta.faces.html"
    xmlns:f="jakarta.faces.core">
<h:head>
  <meta charset="UTF-8"/>
  <title>Iniciar SesiÃ³n - Conversor SOAP (Java Faces)</title>
</h:head>
<h:body>

  <div align="center">
    <h:form id="loginForm">
      <h:panelGrid columns="1" style="text-align: center; margin-top: 50px; border: 1px solid #999; padding: 25px; border-radius: 8px; width: 380px;">

        <h:graphicImage value="/images/sullivan.png" width="120" height="120" alt="Sullivan Monsters Inc" />

        <h:outputText value="INICIAR SESIÃ“N" style="font-size: 20px; font-weight: bold; margin: 15px 0; display: block;" />

        <h:messages globalOnly="true" style="color: red; list-style-type: none; padding: 0;" />

        <h:panelGrid columns="2" cellpadding="8" style="text-align: left; width: 100%;">
          <h:outputLabel for="usuario" value="Usuario:" style="font-weight: bold;" />
          <h:inputText id="usuario" value="#{loginView.usuario}" required="true" requiredMessage="Ingrese su usuario." style="width: 100%;" />

          <h:outputLabel for="contrasenia" value="ContraseÃ±a:" style="font-weight: bold;" />
          <h:inputSecret id="contrasenia" value="#{loginView.contrasenia}" required="true" requiredMessage="Ingrese su contraseÃ±a." style="width: 100%;" />
        </h:panelGrid>

        <h:panelGroup layout="block" style="margin-top: 15px;">
          <h:commandButton value="Ingresar" action="#{loginView.login}" style="padding: 6px 16px; margin-right: 10px; cursor: pointer;" />
          <h:commandButton value="Limpiar" action="#{loginView.limpiar}" immediate="true" style="padding: 6px 16px; cursor: pointer;">
            <f:ajax execute="@this" render="loginForm" />
          </h:commandButton>
        </h:panelGroup>

      </h:panelGrid>
```

```
    </h:form>
  </div>

</h:body>
</html>
```

Tabla 30

Interfaz Facelets del conversor web reactivo (conversor.xhtml)

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="jakarta.faces.html"
    xmlns:f="jakarta.faces.core">

<f:metadata>
  <f:viewAction action="#{conversorView.verificarSesion}" />
</f:metadata>

<h:head>
  <meta charset="UTF-8"/>
  <title>#{conversorView.titulo} - Java Faces</title>
</h:head>
<h:body>

  <!-- Barra superior con usuario y salir -->
  <h:form id="navForm">
    <h:panelGrid columns="2" style="width: 100%; border-bottom: 1px solid #ccc; padding:
10px 20px;">
      <h:panelGroup>
        <h:graphicImage value="/images/sullivan.png" width="30" height="30"
style="vertical-align: middle; margin-right: 8px;" />
        <h:outputText value="WS Conversor de Unidades (Jakarta Faces)" style="font-
weight: bold; font-size: 16px;" />
      </h:panelGroup>
      <h:panelGroup style="text-align: right; display: block;">
        <h:outputText value="Usuario: " />
        <h:outputText value="#{loginView.nombreUsuario}" style="font-weight: bold;
margin-right: 15px;" />
        <h:commandLink value="Cerrar SesiÃ³n" action="#{conversorView.logout}"
style="color: #c00; font-weight: bold;" />
      </h:panelGroup>
    </h:panelGrid>
  </h:form>

  <div align="center">
    <h:form id="conversorForm">
      <h:panelGrid columns="1" style="text-align: center; margin-top: 30px; border: 1px
solid #999; padding: 25px; border-radius: 8px; width: 500px;">

        <h:graphicImage value="/images/sullivan.png" width="80" height="80" alt="Sullivan
Monsters Inc" />
```

```
        <h:outputText value="#{conversorView.titulo}" style="font-size: 20px; font-weight:
bold; margin: 10px 0; display: block;" />

        <h:messages globalOnly="true" style="color: red; list-style-type: none; padding: 0;"
/>

        <h:panelGrid columns="2" cellpadding="8" style="text-align: left; width: 100%;">
          <h:outputLabel for="cmbMagnitud" value="Tipo de Magnitud:" style="font-
weight: bold;" />
          <h:selectOneMenu id="cmbMagnitud" value="#{conversorView.tipoMagnitud}"
style="width: 100%;">
            <f:selectItem itemValue="LONGITUD" itemLabel="Longitud" />
            <f:selectItem itemValue="MASA" itemLabel="Masa" />
            <f:selectItem itemValue="TEMPERATURA" itemLabel="Temperatura" />
            <f:ajax listener="#{conversorView.cambiarMagnitud}" render="conversorForm"
/>
          </h:selectOneMenu>

          <h:outputLabel for="txtValor" value="Valor:" style="font-weight: bold;" />
          <h:inputText id="txtValor" value="#{conversorView.valor}" style="width: 100%;"
/>

          <h:outputLabel for="cmbOrigen" value="De:" style="font-weight: bold;" />
          <h:selectOneMenu id="cmbOrigen" value="#{conversorView.unidadOrigen}"
style="width: 100%;">
            <f:selectItems value="#{conversorView.unidadesDisponibles}" />
          </h:selectOneMenu>

          <h:outputLabel for="cmbDestino" value="A:" style="font-weight: bold;" />
          <h:selectOneMenu id="cmbDestino" value="#{conversorView.unidadDestino}"
style="width: 100%;">
            <f:selectItems value="#{conversorView.unidadesDisponibles}" />
          </h:selectOneMenu>

          <h:outputLabel for="txtResultado" value="Resultado:" style="font-weight: bold;"
/>
          <h:inputText id="txtResultado" value="#{conversorView.resultado}"
readonly="true" style="width: 100%; font-weight: bold; color: green;" />
        </h:panelGrid>

        <h:panelGroup layout="block" style="margin: 20px 0;">
          <h:commandButton value="Convertir" action="#{conversorView.convertir}"
style="padding: 6px 18px; margin-right: 10px; cursor: pointer;">
            <f:ajax execute="@form" render="conversorForm" />
          </h:commandButton>
          <h:commandButton value="Limpiar" action="#{conversorView.limpiar}"
immediate="true" style="padding: 6px 18px; margin-right: 10px; cursor: pointer;">
            <f:ajax execute="@this" render="conversorForm" />
          </h:commandButton>
```

```
            <h:commandButton value="Cerrar" action="#{conversorView.logout}"
immediate="true" style="padding: 6px 18px; cursor: pointer;" />
          </h:panelGroup>

          <!-- Historial con h:dataTable nativo de JSF -->
          <h:panelGroup layout="block" rendered="#{not empty conversorView.historial}"
style="margin-top: 15px; width: 100%;">
            <h:outputText value="Historial de Conversiones" style="font-weight: bold; display:
block; margin-bottom: 8px;" />
            <h:dataTable value="#{conversorView.historial}" var="item" border="1"
cellpadding="6" cellspacing="0" style="width: 100%; text-align: center; border-collapse:
collapse;">
              <h:column>
                <f:facet name="header">Magnitud</f:facet>
                #{item.magnitud}
              </h:column>
              <h:column>
                <f:facet name="header">Origen</f:facet>
                #{item.valorOrigen} #{item.unidadOrigen}
              </h:column>
              <h:column>
                <f:facet name="header">Destino</f:facet>
                <strong>#{item.valorDestino} #{item.unidadDestino}</strong>
              </h:column>
            </h:dataTable>
          </h:panelGroup>

        </h:panelGrid>
      </h:form>
    </div>

</h:body>
</html>
```

Figura 33

Interfaz Web de Login en navegador (login.xhtml)

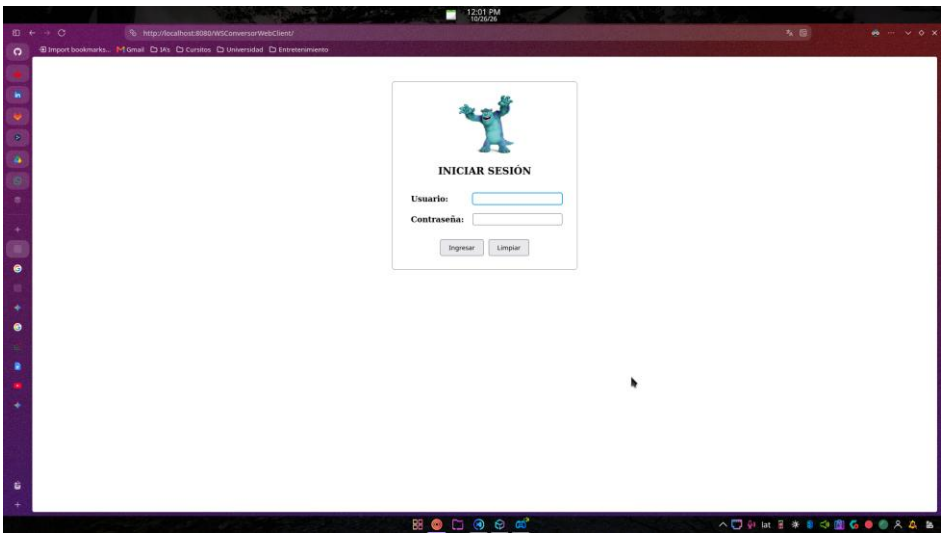
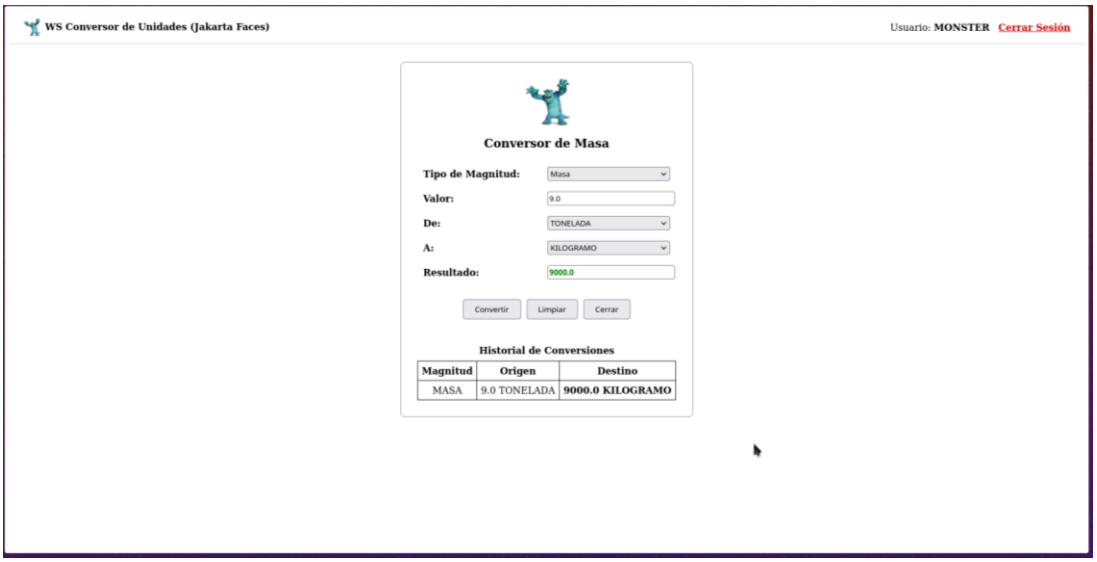


Figura 34
Interfaz Web del Conversor con actualización reactiva Ajax (conversor.xhtml)



5 CONCLUSIONES

Se demostró que la arquitectura RESTful basada en recursos y el protocolo HTTP proporciona un nivel superior de agilidad, interoperabilidad y ligereza en comparación con los esquemas SOAP tradicionales. La adopción de JSON como formato de intercambio redujo la sobrecarga de datos en las peticiones y permitió que cuatro clientes nativos heterogéneos (consola, escritorio, web y móvil) consumieran el servicio sin requerir la compilación ni descarga de contratos formales WSDL.

La especificación Jakarta REST 3.1 (JAX-RS) sobre Payara Server 6 demostró ser una solución empresarial de alto rendimiento. Las anotaciones declarativas (@ApplicationPath, @Path, @GET, @POST) y el contenedor de inyección CDI permitieron desacoplar limpiamente los endpoints de transporte de la lógica de dominio físico y matemático.

El mecanismo de autenticación y seguridad en memoria sin base de datos (SIN BDD) sustentado en Bearer Tokens y colecciones concurrentes (ConcurrentHashMap) resolvió satisfactoriamente los requerimientos de gobernanza y control de acceso. La utilización de la cabecera estándar HTTP Authorization y filtros perimetrales CORS garantizó la seguridad perimetral de forma homogénea en las cuatro plataformas cliente.

La implementación rigurosa del patrón arquitectónico Modelo-Vista-Controlador (MVC) en exactamente tres paquetes (model, view, controller) en todos los clientes demostró que es posible mantener una arquitectura limpia y desacoplada sin recurrir a capas intermedias innecesarias. Cada cliente encapsuló el protocolo REST en su respectiva clase ConversorService, permitiendo que las vistas y controladores se concentraran exclusivamente en la experiencia del usuario.

El soporte multiplataforma se validó exitosamente: el cliente de consola interactivo con banner ASCII, el cliente de escritorio con Java Swing, el cliente web en Jakarta Faces con 0% JavaScript externo y 100% Java Facelets reactivo, y el cliente móvil nativo en Android con configuración de IP dinámica demostraron total interoperabilidad al consumir los mismos recursos RESTful centralizados.

6 RECOMENDACIONES

En fases posteriores de evolución del sistema, se recomienda implementar una capa de persistencia formal mediante una base de datos relacional (como PostgreSQL o MySQL) con

Jakarta Persistence API (JPA / Hibernate), reemplazando el almacenamiento en memoria volátil de credenciales para habilitar auditoría persistente, historial de transacciones y roles de usuario avanzados.
Configurar el protocolo de cifrado en tránsito HTTPS/TLS en el servidor Payara Server 6 para garantizar la total confidencialidad e integridad del Bearer Token y los datos intercambiados en redes de comunicación abiertas.
Incorporar la especificación OpenAPI 3.0 / Swagger en el servidor RESTful para generar automáticamente documentación interactiva y esquemas de prueba accesibles mediante interfaz web Swagger UI.
Ejecutar pruebas automatizadas de rendimiento y estrés de carga concurrente utilizando herramientas como Apache JMeter para dimensionar la capacidad de respuesta del servidor Payara ante miles de peticiones simultáneas de conversión.

7 REFERENCIAS BIBLIOGRÁFICAS

1. Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures. Doctoral dissertation, University of California, Irvine.
2. Fowler, M. (2006). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.
3. Jakarta EE Working Group. (2026). Jakarta RESTful Web Services 3.1 Specification. Eclipse Foundation. Recuperado de los archivos del proyecto.
4. Payara Services. (2026). Payara Server Enterprise Documentation: Jakarta REST and Cloud Architecture. Recuperado de los archivos del proyecto.
5. Red Hat. (2026). Arquitectura de APIs RESTful y microservicios empresariales. Red Hat Documentation. Recuperado de los archivos del proyecto.
6. Richardson, L., & Ruby, S. (2007). RESTful Web Services. O'Reilly Media.